

# Formal Verification of the Number-Theoretic Transform in the Haetae Post-Quantum Signature Scheme

A Dissertation Submitted in Partial Fulfillment  
of the Requirements for the Degree of  
**Doctor of Philosophy**

*Author Name*

Department of Computer Science and Engineering  
Institution Name

---

Supervised by:

**Supervisor Name**

Professor, Department of Computer Science and Engineering

---

---

2026

This dissertation is submitted under the license of [CC BY 4.0 International](#)

# Abstract

The transition to post-quantum cryptography demands not only mathematically sound security arguments but also *machine-verified* correctness proofs for concrete implementations. This dissertation presents a comprehensive formal verification of the Number-Theoretic Transform (NTT) at the core of the HAETAE lattice-based digital signature scheme—a Korean national standard candidate under the KPQC competition.

We establish functional correctness across three vertically composed abstraction layers. At the *implementation layer*, the high-assurance **Jasmin** programming language produces x86-64 assembly that admits direct extraction into **EasyCrypt** modules. At the *algorithmic layer*, an imperative specification in **EasyCrypt** faithfully models the in-place Cooley–Tukey butterfly structure together with Montgomery modular arithmetic over  $\mathbb{Z}_q$ ,  $q = 64513$ , including tight coefficient-bound invariants across all eight butterfly stages. At the *mathematical layer*, a fully abstract spectral specification expresses the NTT as a Vandermonde-style evaluation at powers of a primitive 512th root of unity in  $\mathbb{Z}_q$ .

The key technical contributions are:

1. A machine-checked **Montgomery reduction correctness** proof, showing that `montgomery_reduce(a)`  $\equiv a \cdot R^{-1} \pmod{q}$  for all  $a$  in the 16-bit to 32-bit working range, with explicit arithmetic bit-width bounds.
2. A **butterfly-stage induction** connecting the iterative forward NTT loop in `NTT_Fq.ec` to the closed-form evaluation formula in `NTTFullSpec.ec`, parameterised over the 8-level Cooley–Tukey recursion for the negacyclic polynomial ring  $\mathbb{Z}_q[X]/(X^{256} + 1)$ .
3. An **inverse NTT correctness** proof that tracks the Montgomery scaling factor  $R^{-1} \pmod{q}$  introduced by the final output normalisation step, and

---

establishes  $\text{invntt\_tomont} \circ \text{ntt} = \text{id}$  on valid coefficient representations.

4. A **Jasmin-to-EasyCrypt refinement** proof in `RefJasminNTT.ec` connecting the extracted Jasmin semantics to the imperative specifications, with a 16-bit-to-24-bit coefficient-bound analysis for the forward transform and a tighter 16-bit preservation proof for the inverse transform.
5. An **end-to-end statement** in `NTTEndToEnd.ec` asserting that the Jasmin exported functions `poly_ntt_jazz` and `poly_invntt_jazz` compute exactly the abstract spectral NTT and its inverse, bringing the entire verification chain under a single top-level theorem.

The verification identifies a critical structural difference between the HAETAE NTT and the MLKEM NTT: the former performs a *complete* 256-point negacyclic transform with bit-reversal exponent  $\text{brev}_8(s \bmod \ell)$ , whereas the latter applies an incomplete 128-point transform with exponent  $\text{brev}_8((s \bmod \ell) \cdot 2)$ . This difference invalidates a naive reuse of existing MLKEM algebra lemmas and motivates new algebraic infrastructure developed in `NTTFullAlgebra.ec`.

All proofs are machine-checked in EasyCrypt version 2024 on an x86-64 Linux platform. The complete development totals approximately 3 500 lines of EasyCrypt and 200 lines of Jasmin code.

**Keywords:** Number-Theoretic Transform, Formal Verification, EasyCrypt, Jasmin, Post-Quantum Cryptography, HAETAE, Montgomery Arithmetic, Lattice-Based Signatures, Functional Correctness, Negacyclic Convolution.

# Acknowledgements

I am deeply grateful to my supervisor, **Supervisor Name**, whose insight into formal methods and post-quantum cryptography shaped every aspect of this work. Their patience in guiding me through the intricacies of **EasyCrypt**’s pRHL calculus and the semantic model of **Jasmin** was invaluable.

I thank the members of the HAETAE design team—in particular the authors of the KPQC submission—for open-sourcing the reference C implementation and for helpful discussions about the mathematical structure of the negacyclic NTT.

I am indebted to the **EasyCrypt** development team for producing a tool that makes large-scale cryptographic proofs tractable, and to the **Jasmin** compiler team for the extraction infrastructure that forms the bridge between implementation and specification.

Financial support was provided by Funding Agency / Grant Number.

City, Month 2026

*Author Name*



# Contents

|                                                  |            |
|--------------------------------------------------|------------|
| <b>Abstract</b>                                  | <b>iii</b> |
| <b>Acknowledgements</b>                          | <b>v</b>   |
| <b>1 Introduction</b>                            | <b>1</b>   |
| 1.1 Motivation . . . . .                         | 1          |
| 1.2 Problem Statement . . . . .                  | 2          |
| 1.3 Tools and Methodology . . . . .              | 3          |
| 1.4 Main Contributions . . . . .                 | 3          |
| 1.5 Structure of the Dissertation . . . . .      | 4          |
| <b>2 Background</b>                              | <b>7</b>   |
| 2.1 Lattice-Based Cryptography . . . . .         | 7          |
| 2.1.1 Module Lattices . . . . .                  | 7          |
| 2.1.2 NTT-Friendly Parameters . . . . .          | 7          |
| 2.2 Number-Theoretic Transform . . . . .         | 8          |
| 2.2.1 Definition . . . . .                       | 8          |
| 2.2.2 Cooley–Tukey Butterfly Algorithm . . . . . | 8          |
| 2.2.3 Negacyclic NTT vs. Standard NTT . . . . .  | 9          |
| 2.3 Montgomery Modular Arithmetic . . . . .      | 9          |

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| 2.3.1    | REDC Algorithm . . . . .                            | 9         |
| 2.3.2    | Montgomery Multiplication . . . . .                 | 10        |
| 2.3.3    | Signed Montgomery Reduction . . . . .               | 10        |
| 2.4      | The EasyCrypt Proof Assistant . . . . .             | 10        |
| 2.4.1    | Architecture . . . . .                              | 10        |
| 2.4.2    | Hoare Logic in EasyCrypt . . . . .                  | 11        |
| 2.4.3    | Module System . . . . .                             | 11        |
| 2.4.4    | Theory Cloning . . . . .                            | 11        |
| 2.5      | The Jasmin Programming Language . . . . .           | 11        |
| 2.6      | Related Work . . . . .                              | 12        |
| <b>3</b> | <b>The Haetae Signature Scheme and Its NTT</b>      | <b>15</b> |
| 3.1      | Overview of HAETAETAE . . . . .                     | 15        |
| 3.1.1    | Algebraic Setting . . . . .                         | 15        |
| 3.1.2    | Role of the NTT . . . . .                           | 16        |
| 3.2      | HAETAETAE NTT Parameters . . . . .                  | 16        |
| 3.2.1    | Twiddle Factors . . . . .                           | 16        |
| 3.3      | Reference C Implementation . . . . .                | 17        |
| 3.3.1    | Forward NTT . . . . .                               | 17        |
| 3.3.2    | Inverse NTT . . . . .                               | 18        |
| 3.3.3    | Montgomery Reduction . . . . .                      | 19        |
| 3.3.4    | Test Harness . . . . .                              | 19        |
| 3.4      | Structural Comparison with MLKEM NTT . . . . .      | 20        |
| <b>4</b> | <b>Formal Verification Framework</b>                | <b>21</b> |
| 4.1      | Overview of the Verification Architecture . . . . . | 21        |

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| 4.2      | Layer 0: Jasmin Source . . . . .                      | 22        |
| 4.3      | Layer 1: Extracted EasyCrypt Module . . . . .         | 22        |
| 4.4      | Layer 2: Imperative EasyCrypt Specification . . . . . | 23        |
| 4.5      | Layer 3: Abstract Spectral Specification . . . . .    | 23        |
| 4.6      | End-to-End Composition . . . . .                      | 24        |
| 4.7      | Proof Methodology . . . . .                           | 24        |
| 4.7.1    | Proceeding Bottom-Up . . . . .                        | 24        |
| 4.7.2    | Hoare Logic for Loop Invariants . . . . .             | 25        |
| 4.7.3    | pRHL for Procedure Equivalence . . . . .              | 25        |
| 4.7.4    | Field-Theory Automation . . . . .                     | 25        |
| 4.8      | File Dependency Graph . . . . .                       | 26        |
| 4.9      | Relationships Between the EasyCrypt Files . . . . .   | 26        |
| <b>5</b> | <b>Formalisation of Field and Ring Arithmetic</b>     | <b>31</b> |
| 5.1      | The Prime Field $\mathbb{F}_q$ . . . . .              | 31        |
| 5.1.1    | Field Definition in <code>Fq.ec</code> . . . . .      | 31        |
| 5.1.2    | Signed vs. Unsigned Representation . . . . .          | 32        |
| 5.1.3    | Bit-Width Bounds . . . . .                            | 32        |
| 5.1.4    | Signed Arithmetic Lemmas . . . . .                    | 32        |
| 5.2      | The $\mathbb{Z}_q$ Field via Theory Cloning . . . . . | 33        |
| 5.2.1    | <code>GFq.ec</code> : Abstract Field . . . . .        | 33        |
| 5.2.2    | Primitive Root . . . . .                              | 33        |
| 5.3      | The Polynomial Ring $R_q$ . . . . .                   | 34        |
| 5.3.1    | <code>Rq.ec</code> : Ring Definition . . . . .        | 34        |
| 5.3.2    | NTT Axioms in <code>Rq.ec</code> . . . . .            | 35        |
| 5.4      | Array Infrastructure . . . . .                        | 35        |

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| 5.4.1    | Array256.ec . . . . .                                        | 35        |
| 5.4.2    | Word Array Conversion . . . . .                              | 36        |
| 5.5      | Reduction and Auxiliary Operations . . . . .                 | 36        |
| 5.5.1    | Conditional Addition (caddq) . . . . .                       | 36        |
| 5.5.2    | Freeze (freeze) . . . . .                                    | 37        |
| <b>6</b> | <b>Formalisation of Montgomery Arithmetic</b>                | <b>39</b> |
| 6.1      | Motivation and Design Choices . . . . .                      | 39        |
| 6.2      | Montgomery Parameters . . . . .                              | 39        |
| 6.3      | Signed Montgomery Reduction . . . . .                        | 40        |
| 6.3.1    | Algorithm . . . . .                                          | 40        |
| 6.3.2    | Mathematical Analysis . . . . .                              | 41        |
| 6.3.3    | EasyCrypt Formalisation . . . . .                            | 41        |
| 6.4      | Montgomery Multiplication (fqmul) . . . . .                  | 42        |
| 6.5      | Twiddle Factor Multiplication in the NTT Butterfly . . . . . | 42        |
| 6.6      | Bound Tracking Through NTT Stages . . . . .                  | 43        |
| <b>7</b> | <b>Imperative NTT Specification</b>                          | <b>45</b> |
| 7.1      | Overview of NTT_Fq.ec . . . . .                              | 45        |
| 7.2      | Twiddle Factor Table . . . . .                               | 45        |
| 7.3      | Forward NTT Procedure . . . . .                              | 46        |
| 7.3.1    | Specification . . . . .                                      | 46        |
| 7.3.2    | Loop Structure Analysis . . . . .                            | 47        |
| 7.3.3    | Loop Invariants . . . . .                                    | 47        |
| 7.3.4    | Termination . . . . .                                        | 48        |
| 7.4      | Inverse NTT Procedure . . . . .                              | 48        |

|          |                                                                                      |           |
|----------|--------------------------------------------------------------------------------------|-----------|
| 7.4.1    | Specification . . . . .                                                              | 48        |
| 7.4.2    | Differences from the Forward NTT . . . . .                                           | 49        |
| 7.4.3    | Final Scaling . . . . .                                                              | 50        |
| 7.5      | Round-Trip Theorem at the Imperative Level . . . . .                                 | 50        |
| 7.6      | Twiddle Factor Synchronisation . . . . .                                             | 51        |
| <b>8</b> | <b>Algebraic NTT Infrastructure</b>                                                  | <b>53</b> |
| 8.1      | Overview of <code>NTTFullSpec.ec</code> and <code>NTTFullAlgebra.ec</code> . . . . . | 53        |
| 8.2      | Primitive Root Properties . . . . .                                                  | 53        |
| 8.2.1    | Existence and Uniqueness . . . . .                                                   | 53        |
| 8.2.2    | Orthogonality . . . . .                                                              | 54        |
| 8.3      | Bit-Reversal Permutation . . . . .                                                   | 54        |
| 8.3.1    | Definition . . . . .                                                                 | 54        |
| 8.3.2    | Key Properties . . . . .                                                             | 55        |
| 8.3.3    | Bit-Reversal and Twiddle Factor Indexing . . . . .                                   | 55        |
| 8.4      | Stage Decomposition Theorem . . . . .                                                | 56        |
| 8.4.1    | Partial NTT . . . . .                                                                | 56        |
| 8.4.2    | Butterfly Stage Lemma . . . . .                                                      | 56        |
| 8.4.3    | Full NTT Correctness . . . . .                                                       | 57        |
| 8.5      | Inverse NTT Algebra . . . . .                                                        | 57        |
| 8.5.1    | Abstract Inverse NTT . . . . .                                                       | 57        |
| 8.6      | The Critical Difference from MLKEM . . . . .                                         | 58        |
| <b>9</b> | <b>Jasmin Implementation</b>                                                         | <b>59</b> |
| 9.1      | Overview . . . . .                                                                   | 59        |
| 9.2      | Montgomery Reduction in Jasmin . . . . .                                             | 59        |

|           |                                                    |           |
|-----------|----------------------------------------------------|-----------|
| 9.2.1     | Source Code . . . . .                              | 59        |
| 9.2.2     | Jasmin Semantics . . . . .                         | 60        |
| 9.2.3     | Correspondence to C . . . . .                      | 60        |
| 9.3       | Field Multiplication . . . . .                     | 61        |
| 9.4       | Forward NTT Kernel . . . . .                       | 61        |
| 9.4.1     | Source Code . . . . .                              | 61        |
| 9.4.2     | Structural Correspondence . . . . .                | 62        |
| 9.4.3     | Array Access Pattern . . . . .                     | 63        |
| 9.5       | Inverse NTT Kernel . . . . .                       | 63        |
| 9.5.1     | Inverse Butterfly Structure . . . . .              | 64        |
| 9.6       | Exported Entry Points . . . . .                    | 64        |
| 9.7       | Safety Verification . . . . .                      | 65        |
| 9.8       | Generated Assembly . . . . .                       | 65        |
| <b>10</b> | <b>Jasmin-to-EasyCrypt Refinement Proof</b>        | <b>67</b> |
| 10.1      | Overview of <code>RefJasminNTT.ec</code> . . . . . | 67        |
| 10.2      | Word-to-Coefficient Correspondence . . . . .       | 67        |
| 10.2.1    | The Linking Predicate . . . . .                    | 67        |
| 10.2.2    | Preservation Under Arithmetic . . . . .            | 68        |
| 10.3      | Montgomery Reduction Correspondence . . . . .      | 68        |
| 10.3.1    | Word-Level Reduction . . . . .                     | 68        |
| 10.4      | Butterfly Equivalence . . . . .                    | 69        |
| 10.4.1    | Individual Butterfly . . . . .                     | 69        |
| 10.5      | Loop Equivalence Proofs . . . . .                  | 69        |
| 10.5.1    | Inner Loop Equivalence . . . . .                   | 69        |
| 10.5.2    | Full NTT Loop Equivalence . . . . .                | 70        |

|           |                                                   |           |
|-----------|---------------------------------------------------|-----------|
| 10.6      | Main Refinement Lemmas . . . . .                  | 70        |
| 10.7      | Proof Engineering Challenges . . . . .            | 71        |
| 10.7.1    | Signed vs. Unsigned Arithmetic . . . . .          | 71        |
| 10.7.2    | 64-bit Product Non-Overflow . . . . .             | 71        |
| 10.7.3    | Twiddle Factor Table Agreement . . . . .          | 71        |
| 10.7.4    | In-Place Update Semantics . . . . .               | 72        |
| <b>11</b> | <b>End-to-End Correctness</b>                     | <b>73</b> |
| 11.1      | Overview of <code>NTTEndToEnd.ec</code> . . . . . | 73        |
| 11.2      | Main Correctness Theorems . . . . .               | 73        |
| 11.2.1    | Forward NTT Correctness . . . . .                 | 73        |
| 11.2.2    | Inverse NTT Correctness . . . . .                 | 74        |
| 11.3      | Round-Trip Theorem . . . . .                      | 75        |
| 11.4      | Proof Composition . . . . .                       | 76        |
| 11.5      | Status of the Proof . . . . .                     | 76        |
| 11.5.1    | Completed Machine-Checked Steps . . . . .         | 76        |
| 11.5.2    | Remaining Proof Obligation . . . . .              | 77        |
| 11.6      | Summary of Verification Metrics . . . . .         | 77        |
| <b>12</b> | <b>Conclusion</b>                                 | <b>79</b> |
| 12.1      | Summary of Contributions . . . . .                | 79        |
| 12.2      | Limitations and Remaining Obligations . . . . .   | 80        |
| 12.2.1    | Stage Induction Gap . . . . .                     | 80        |
| 12.2.2    | No AVX2 Vectorisation . . . . .                   | 80        |
| 12.2.3    | Base Multiplication Not Fully Verified . . . . .  | 81        |
| 12.2.4    | Security Proof Integration . . . . .              | 81        |

|                                        |           |
|----------------------------------------|-----------|
| 12.3 Future Work . . . . .             | 81        |
| 12.4 Closing Remarks . . . . .         | 82        |
| <b>A Visualisation Gallery</b>         | <b>83</b> |
| <b>Appendix: Visualisation Gallery</b> | <b>83</b> |
| <b>Bibliography</b>                    | <b>89</b> |

# List of Figures

|     |                                                                                                                                                                                                                                                                                                                                                                           |    |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.1 | Four-layer verification architecture for the HAETAE NTT. Solid arrows are machine-checked proof steps; dashed arrows indicate the end-to-end composition. . . . .                                                                                                                                                                                                         | 21 |
| 4.2 | Core EasyCrypt file dependency graph for the HAETAE NTT verification. Arrows point from dependency to dependent; low-level Jasmin array clones and standard-library theories are omitted here and recorded in Table 4.1. . . . .                                                                                                                                          | 26 |
| A.1 | Complete four-layer formal verification architecture for the HAETAE NTT. Solid arrows are machine-checked proof steps. Dashed arrows show the end-to-end composition in <code>NTTEndToEnd.ec</code> . . . . .                                                                                                                                                             | 83 |
| A.2 | One stage of the Cooley–Tukey butterfly for $n = 8$ (shown for clarity; the actual NTT uses $n = 256$ over 8 stages). Red paths carry the $+t$ update to the upper half; blue paths carry the $-t$ update to the lower half. Each pair of red+blue paths forms one butterfly using twiddle factor $w_k = \zeta_0^{\text{brevs}(k)} \cdot R \bmod q$ . . . . .             | 84 |
| A.3 | Data-flow view of the eight-stage forward NTT. Each column is one outer-loop iteration ( $\ell$ value). Green cells are groups fully processed; white cells are untouched. The coefficient bound grows by one bit per stage: input $< 2^{15}$ , output $< 2^{23}$ . The twiddle counter column ( $k+$ ) shows how many twiddle factors have been consumed cumulatively. . | 84 |
| A.4 | Dataflow diagram of the signed Montgomery reduction algorithm. The two 32-bit multiplications ( <code>QINV</code> and <code>q</code> ) and the arithmetic right shift by 32 are the three core operations. The key divisibility invariant ensures that the right shift is exact (no remainder), giving the congruence $t \equiv aR^{-1} \pmod{q}$ . . . . .               | 85 |

|      |                                                                                                                                                                                                                                                                                                                                                                                                                                   |    |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| A.5  | The negacyclic polynomial ring $R_q$ and its isomorphism to the NTT evaluation domain. The NTT evaluates the polynomial at all 256 primitive 512th roots of unity (red dots on the unit circle). This is an instance of the Chinese Remainder Theorem decomposition of $R_q$ over $\mathbb{F}_q$ . . . . .                                                                                                                        | 85 |
| A.6  | The bit-reversal permutation $\text{brev}_8$ for the first 16 loop-counter values $k = 0, \dots, 15$ . Red cells in the bit-pattern boxes highlight the set bits. The bottom row gives the Montgomery-form twiddle factor $\zeta_0^{\text{brev}_8(k)} \cdot R \bmod q$ stored in <code>jzetas[k]</code> . For the forward NTT, the inner loop uses <code>jzetas[k]</code> as the twiddle factor for butterfly group $k$ . . . . . | 86 |
| A.7  | Structure of the <b>EasyCrypt</b> proof judgements. The topmost judgement (pRHL) relates machine-word arrays to integer arrays. The middle (Hoare) judgement captures the loop semantics in integer arithmetic. The algebraic equality connects the loop output to the closed-form spectral sum. The bottom (end-to-end) judgement composes all three. . . . .                                                                    | 86 |
| A.8  | Structural comparison of the HAETAΕ and MLKEM NTT. Green box: aspects where methodology is transferable between the two verification projects. Red box: the critical structural difference that prevents direct reuse of MLKEM algebra lemmas. . . . .                                                                                                                                                                            | 87 |
| A.9  | Coefficient bound propagation across all four abstraction layers for both the forward and inverse NTT, and the round-trip composition. Each layer has its own type of bound expression: machine-word arithmetic bounds at the Jasmin level, integer bounds at the imperative level, field elements at the algebraic level. . . . .                                                                                                | 87 |
| A.10 | Detailed <b>EasyCrypt</b> module dependency graph. Arrows point from dependency to dependent file; standard-library and some transitive edges are omitted for readability. Nodes are colour-coded by layer: orange = foundations, yellow = imperative specification, green = algebraic specification, red = refinement, cyan = end-to-end. . . . .                                                                                | 88 |

# List of Tables

|      |                                                                                  |    |
|------|----------------------------------------------------------------------------------|----|
| 3.1  | HAETAE NTT parameters . . . . .                                                  | 16 |
| 3.2  | First sixteen twiddle factors <code>zetas[1..16]</code> (in Montgomery form) . . | 17 |
| 3.3  | Comparison of HAETAE and MLKEM NTT structures . . . . .                          | 20 |
| 4.1  | Relationships between the EasyCrypt source files . . . . .                       | 27 |
| 6.1  | Coefficient bounds at each layer of the verification . . . . .                   | 44 |
| 11.1 | Verification metrics by file . . . . .                                           | 77 |



# Listings

|     |                                                                            |    |
|-----|----------------------------------------------------------------------------|----|
| 2.1 | Jasmin function signature example . . . . .                                | 12 |
| 3.1 | Forward NTT from <code>ntt.c</code> . . . . .                              | 17 |
| 3.2 | Inverse NTT from <code>ntt.c</code> . . . . .                              | 18 |
| 3.3 | Montgomery reduction from <code>reduce.c</code> . . . . .                  | 19 |
| 4.1 | Fragment of extracted procedure in <code>Hpoly_extract.ec</code> . . . . . | 22 |
| 4.2 | End-to-end theorem statement in <code>NTTEndToEnd.ec</code> . . . . .      | 24 |
| 5.1 | Field declaration in <code>Fq.ec</code> . . . . .                          | 31 |
| 5.2 | <code>GFq.ec</code> : Galois field instantiation . . . . .                 | 33 |
| 5.3 | Primitive root declaration in <code>NTTFullAlgebra.ec</code> . . . . .     | 34 |
| 5.4 | Ring definition in <code>Rq.ec</code> . . . . .                            | 34 |
| 5.5 | NTT homomorphism axioms in <code>Rq.ec</code> . . . . .                    | 35 |
| 6.1 | Signed Montgomery reduction (C reference) . . . . .                        | 40 |
| 6.2 | Montgomery reduction specification in <code>Fq.ec</code> . . . . .         | 41 |
| 6.3 | <code>fqmul</code> specification . . . . .                                 | 42 |
| 7.1 | Forward NTT specification in <code>NTT_Fq.ec</code> . . . . .              | 46 |
| 7.2 | Inverse NTT specification in <code>NTT_Fq.ec</code> . . . . .              | 48 |
| 8.1 | Bit-reversal definition in <code>NTTFullAlgebra.ec</code> . . . . .        | 54 |
| 9.1 | Montgomery reduction in <code>reduce.jinc</code> . . . . .                 | 59 |
| 9.2 | Field multiplication in <code>reduce.jinc</code> . . . . .                 | 61 |

|      |                                                                          |    |
|------|--------------------------------------------------------------------------|----|
| 9.3  | Forward NTT kernel in <code>poly.jinc</code> . . . . .                   | 61 |
| 9.4  | Inverse NTT kernel in <code>poly.jinc</code> (condensed) . . . . .       | 63 |
| 9.5  | Exported functions in <code>hpoly.jazz</code> . . . . .                  | 64 |
| 10.1 | Array correspondence predicate in <code>RefJasminNTT.ec</code> . . . . . | 67 |
| 10.2 | Main refinement lemmas in <code>RefJasminNTT.ec</code> . . . . .         | 70 |
| 11.1 | Module imports in <code>NTTEndToEnd.ec</code> . . . . .                  | 73 |
| 11.2 | End-to-end NTT correctness statement . . . . .                           | 74 |
| 11.3 | End-to-end inverse NTT correctness statement . . . . .                   | 74 |
| 11.4 | Proof composition in <code>NTTEndToEnd.ec</code> . . . . .               | 76 |

# Chapter 1

## Introduction

### 1.1 Motivation

The threat posed by large-scale quantum computers to today’s public-key infrastructure is no longer a theoretical curiosity. Shor’s algorithm reduces the security of RSA, DSA, and elliptic-curve schemes to polynomial time once a sufficiently powerful quantum computer is available [Sho94]. In response, the U.S. National Institute of Standards and Technology (NIST) ran a multi-year post-quantum cryptography standardisation project, selecting four algorithms for standardisation in 2022–2024 [Nat22a], while the Korean Cryptography Module Validation Programme (KPQC) is pursuing its own national standards [Nat22b].

HAETAE [HAE22] is one of the KPQC signature finalists. It is a lattice-based scheme whose security reduces to the Decisional Module Short Integer Solution (DMSIS) hardness assumption over module lattices. Like all modern lattice-based schemes, HAETAE relies on efficient polynomial multiplication modulo  $X^{256} + 1$  in the ring  $\mathbb{Z}_q[X]/(X^{256} + 1)$ , which in practice is implemented via the *Number-Theoretic Transform* (NTT). The NTT reduces an  $O(n^2)$  schoolbook convolution to  $O(n \log n)$  operations and is the dominant computational primitive in the scheme.

For a cryptographic standard to be trustworthy, correctness proofs cannot stop at pen-and-paper mathematics. History has shown that hand-verified implementations harbour subtle bugs: off-by-one errors in loop bounds, incorrect twiddle factor tables, wrong modular arithmetic, or mishandled Montgomery scaling can silently produce wrong outputs while passing standard test vectors [BL17]. This dissertation takes the position that *machine-checked* formal proofs, linking every line of executable

code to its abstract mathematical specification, are the appropriate gold standard for cryptographic primitives in post-quantum standards.

## 1.2 Problem Statement

We address the following problem:

*Prove, in a machine-checkable formal system, that the **Jasmin** implementation of the NTT and inverse-NTT in HAETAΕ is functionally correct with respect to the abstract mathematical specification of the negacyclic NTT over  $\mathbb{Z}_q[X]/(X^{256} + 1)$ .*

The challenge is to bridge three distinct abstraction levels:

1. **Low-level word arithmetic.** The **Jasmin** source is compiled to x86-64 assembly and works with 32-bit and 64-bit machine words. Correctness at this level means proving arithmetic identities modulo  $2^{32}$  and establishing bounds (e.g., that an intermediate product fits in a 64-bit register without overflow).
2. **Algorithmic (loop) level.** The Cooley–Tukey butterfly algorithm is expressed as a nested three-loop structure with eight stages. Correctness here means showing that after all loops terminate, the array holds the intended Fourier coefficients, using loop invariants expressed in terms of partial NTT computations.
3. **Mathematical (spectral) level.** The NTT is a linear map  $\mathbb{Z}_q^{256} \rightarrow \mathbb{Z}_q^{256}$  defined by evaluation at specific powers of a primitive 512th root of unity. Correctness here means proving that the algorithmic output equals the evaluation formula, and that the inverse NTT is its exact algebraic inverse.

Each layer requires different mathematical machinery: modular arithmetic and bit-vector identities at level 1, loop invariants and array relational assertions at level 2, and field theory with primitive roots at level 3. Composing all three in a single proof assistant is the central technical challenge.

## 1.3 Tools and Methodology

We use two mature high-assurance tools:

**Jasmin** [Alm+17]. A domain-specific programming language and compiler designed for high-assurance cryptographic implementations. **Jasmin** programs are compiled to provably correct x86-64 assembly and simultaneously admit extraction into the **EasyCrypt** proof assistant as first-class program modules. The language provides explicit register and memory annotations that remove ambiguity from the compiler’s optimisation decisions.

**EasyCrypt** [BGB09; Bar+13]. A proof assistant built around the probabilistic Relational Hoare Logic (pRHL) calculus, designed specifically for reasoning about cryptographic constructions. An **EasyCrypt** *program logic* allows Hoare-style reasoning over both deterministic and probabilistic procedures. The ambient logic is a dependent type theory supporting classical higher-order reasoning, enabling rich algebraic developments (ring theory, field theory, group theory) within the same tool.

Our proof strategy follows the *refinement* paradigm:

$$\begin{array}{c} \text{Jasmin} \xrightarrow{\text{extraction}} \text{EC module (RefJasminNTT)} \xrightarrow{\text{pRHL equivalence}} \text{EC imperative spec (NTT\_Fq)} \\ \xrightarrow{\text{loop invariant}} \text{EC abstract spec (NTTFullSpec/Algebra)} \end{array}$$

Each arrow represents a machine-checked proof step; the composition yields the end-to-end theorem stated in `NTTEndToEnd.ec`.

## 1.4 Main Contributions

**C1 – Montgomery correctness.** A machine-checked proof that `montgomery_reduce` computes  $a \cdot R^{-1} \bmod q$  for all inputs in the relevant working range, with  $R = 2^{32}$ ,  $q = 64513$ , explicitly verified bit-width bounds, and signed-integer arithmetic lemmas absent from the **EasyCrypt** standard library (Chapter 6).

**C2 – Imperative NTT specification.** A fully proved **EasyCrypt** model of the forward and inverse NTT loops in `NTT_Fq.ec`, including twiddle-factor correctness, coefficient-bound invariants (16-bit input  $\rightarrow$  24-bit output for forward NTT; 16-bit preservation for inverse NTT), and termination witnesses (Chapter 7).

- C3 – Abstract spectral specification.** An algebraic library `NTTFullAlgebra.ec` establishing properties of the primitive root  $\zeta_0 = 426$  in  $\mathbb{Z}_q$ , bit-reversal permutation lemmas, and a butterfly-stage induction connecting the iterative algorithm to the closed-form evaluation formula (Chapter 8).
- C4 – Jasmin refinement.** A pRHL proof in `RefJasminNTT.ec` showing that the extracted `Jasmin` procedures are observationally equivalent to the imperative specifications up to array representation (Chapter 10).
- C5 – End-to-end correctness.** Top-level theorems `haetae_jasmin_ntt_correct` and `haetae_jasmin_invntt_correct` in `NTTEndToEnd.ec`, together with the round-trip theorem  $\text{INTT} \circ \text{NTT} = \text{id}$  (Chapter 11).

## 1.5 Structure of the Dissertation

The remainder of this dissertation is organised as follows.

**Chapter 2** reviews the relevant background: lattice cryptography, the NTT, post-quantum signature schemes, and the theoretical foundations of formal verification with `EasyCrypt` and `Jasmin`.

**Chapter 3** describes the HAETAETAE scheme, its NTT parameters, and the reference C implementation that serves as the algorithmic ground truth.

**Chapter 4** presents the overall formal framework: the three-layer abstraction stack and the proof methodology.

**Chapter 5** covers the formalization of the field  $\mathbb{Z}_q$  and the polynomial ring  $\mathbb{Z}_q[X]/(X^{256} + 1)$  in `EasyCrypt`.

**Chapter 6** details the Montgomery reduction formalization and its correctness proof.

**Chapter 7** presents the imperative NTT specification in `NTT_Fq.ec`, its invariants, and its termination proof.

**Chapter 8** develops the algebraic infrastructure in `NTTFullAlgebra.ec`: primitive root properties, bit-reversal lemmas, and stage decompositions.

**Chapter 9** describes the `Jasmin` implementation in detail.

**Chapter 10** presents the Jasmin-to-EasyCrypt refinement proof in `RefJasminNTT.ec`.

**Chapter 11** assembles all pieces into the end-to-end correctness theorems.

**Chapter 12** discusses limitations, remaining obligations, related and future work.



# Chapter 2

## Background

### 2.1 Lattice-Based Cryptography

#### 2.1.1 Module Lattices

A *lattice*  $\Lambda \subseteq \mathbb{Z}^n$  is a discrete additive subgroup of  $\mathbb{Z}^n$ ; equivalently, it is the  $\mathbb{Z}$ -span of a basis matrix  $\mathbf{B} \in \mathbb{Z}^{n \times n}$ . The *shortest vector problem* (SVP) asks for the shortest non-zero lattice vector under the Euclidean norm, and is conjectured to be hard for quantum computers.

Module lattices generalise to modules over a polynomial ring. Let  $R = \mathbb{Z}[X]/(X^n + 1)$  and  $R_q = R/qR$ . A *module lattice* of rank  $k$  over  $R$  is a free  $R$ -module  $M \cong R^k$  together with a  $\mathbb{Z}$ -embedding into  $\mathbb{Z}^{kn}$ . The *Module Short Integer Solution* (MSIS) and *Module Learning With Errors* (MLWE) problems over such lattices underpin the security of Dilithium, FALCON, and HAETAE.

#### 2.1.2 NTT-Friendly Parameters

For efficient computation, the ring  $R_q$  must support a fast polynomial multiplication algorithm. If  $q \equiv 1 \pmod{2n}$  then  $X^n + 1$  splits completely over  $\mathbb{F}_q$  and the NTT reduces polynomial multiplication to componentwise multiplication of length- $n$  vectors. If only  $q \equiv 1 \pmod{n}$  (but  $q \equiv 3 \pmod{4}$  or similar), a *negacyclic* NTT can still be defined [LN16].

For HAETAE,  $n = 256$  and  $q = 64513 \equiv 1 \pmod{512}$ . This means a primitive 512th

root of unity  $\zeta_0$  exists in  $\mathbb{F}_q$ , enabling a full 256-point negacyclic NTT.

## 2.2 Number-Theoretic Transform

### 2.2.1 Definition

Let  $q$  be an odd prime with  $q \equiv 1 \pmod{2n}$ , and let  $\zeta \in \mathbb{F}_q^*$  be a primitive  $2n$ th root of unity (so  $\zeta^{2n} = 1$ ,  $\zeta^n = -1$ ). The *negacyclic NTT* of a polynomial  $f = \sum_{j=0}^{n-1} f_j X^j \in R_q$  is the vector  $\hat{f} \in \mathbb{F}_q^n$  defined by

$$\hat{f}_i = \sum_{j=0}^{n-1} f_j \cdot \zeta^{(2\text{brev}_8(i)+1) \cdot j}, \quad i = 0, \dots, n-1, \quad (2.1)$$

where  $\text{brev}_8 : \{0, \dots, 255\} \rightarrow \{0, \dots, 255\}$  is the bit-reversal permutation of an 8-bit index.

The inverse NTT is

$$f_i = \frac{1}{n} \sum_{j=0}^{n-1} \hat{f}_j \cdot \zeta^{-((2\text{brev}_8(j)+1) \cdot i)}, \quad i = 0, \dots, n-1. \quad (2.2)$$

**Proposition 2.1** (Round-trip identity).  $\text{INTT}(\text{NTT}(f)) = f$  for all  $f \in R_q$ .

### 2.2.2 Cooley–Tukey Butterfly Algorithm

The NTT defined by eq. (2.1) can be computed in  $O(n \log n)$  field operations using the Cooley–Tukey decimation-in-time (DIT) algorithm. For the negacyclic case with  $n = 256 = 2^8$ , the algorithm proceeds in 8 stages.

At stage  $s$  (with  $s = 0, 1, \dots, 7$ , from outermost to innermost), the half-length is  $\ell = 2^{7-s}$  and the *twiddle factor* for group  $k$  is  $\zeta_k^{(s)} = \zeta^{\text{brev}_8(k) \cdot \ell}$  where  $k$  ranges over the  $2^s$  groups. The butterfly operation on a pair  $(a, b)$  with twiddle factor  $w$  is:

$$(a', b') = (a + w \cdot b, a - w \cdot b). \quad (2.3)$$

---

**Algorithm 1** Forward NTT (Cooley–Tukey DIT, negacyclic,  $n = 256$ )

---

```

1: procedure NTT( $a[0..255]$ )
2:    $k \leftarrow 1$ 
3:   for  $\ell = 128, 64, 32, 16, 8, 4, 2, 1$  do
4:     for  $start = 0, 2\ell, 4\ell, \dots, 254$  do
5:        $w \leftarrow \zeta^{\text{brev}_8(k)}$   $\triangleright k$ -th twiddle factor
6:        $k \leftarrow k + 1$ 
7:       for  $j = start, \dots, start + \ell - 1$  do
8:          $t \leftarrow w \cdot a[j + \ell] \bmod q$ 
9:          $a[j + \ell] \leftarrow a[j] - t$ 
10:         $a[j] \leftarrow a[j] + t$ 
11:       end for
12:     end for
13:   end for
14: end procedure

```

---

### 2.2.3 Negacyclic NTT vs. Standard NTT

A standard (cyclic) DFT of length  $n$  uses the recurrence for the cyclic ring  $\mathbb{Z}_q[X]/(X^n - 1)$ . The *negacyclic* ring  $\mathbb{Z}_q[X]/(X^n + 1)$  requires a pre-twist by powers of  $\zeta$  (a half-integer shift of the exponent), which accounts for the  $+1$  in the exponent formula of eq. (2.1).

Critically for this dissertation, HAETAE uses a *complete*  $n = 256$  negacyclic NTT with bit-reversal exponent  $\text{brev}_8(k)$ , whereas MLKEM uses an incomplete 128-point transform with exponent  $\text{brev}_7(k)$ . This distinction has direct consequences for which algebraic lemmas can be shared between verification projects.

## 2.3 Montgomery Modular Arithmetic

### 2.3.1 REDC Algorithm

Montgomery reduction [Mon85] is a technique for efficient modular multiplication that avoids expensive division. Given  $a \in \mathbb{Z}$  with  $0 \leq a < q \cdot R$  and integers  $q, R$  with  $\gcd(q, R) = 1$ , the Montgomery product is  $a \cdot R^{-1} \bmod q$ .

In the HAETAE implementation,  $R = 2^{32}$ ,  $q = 64513$ ,  $q' = \text{QINV} = 940508161$  ( $\equiv -q^{-1} \bmod 2^{32}$ ), and  $\text{MONT} = R \bmod q = 14321$ .

---

**Algorithm 2** Montgomery Reduction  $\text{REDC}(a, q, R, q')$

---

**Require:**  $a, R = 2^{32}, q' = -q^{-1} \bmod R$

**Ensure:**  $a \cdot R^{-1} \bmod q$

- 1:  $m \leftarrow (a \bmod R) \cdot q' \bmod R$
  - 2:  $t \leftarrow (a + m \cdot q) \div R$
  - 3: **if**  $t \geq q$  **then return**  $t - q$
  - 4: **else return**  $t$
  - 5: **end if**
- 

### 2.3.2 Montgomery Multiplication

Given  $a, b \in \mathbb{Z}_q$  with both represented in Montgomery form (i.e., as  $a \cdot R \bmod q$  and  $b \cdot R \bmod q$ ), their product in Montgomery form is computed as  $\text{REDC}(a \cdot b)$ , since  $\text{REDC}(aR \cdot bR) = abR^2 \cdot R^{-1} = abR \bmod q$ .

### 2.3.3 Signed Montgomery Reduction

The HAETAE implementation uses a *signed* variant where intermediate values are 32-bit signed integers and the final conditional subtraction is replaced by an arithmetic right shift. Specifically, for  $a \in \mathbb{Z}$  with  $|a| < q \cdot 2^{15}$ :

$$\text{int32}(a_{\text{lo}} \cdot q') \cdot q \text{ (as signed 64-bit)} + a \text{ shifted right by 32} \quad (2.4)$$

yields a value congruent to  $a \cdot R^{-1} \pmod{q}$  and bounded by  $q/2$ .

## 2.4 The EasyCrypt Proof Assistant

### 2.4.1 Architecture

EasyCrypt [BGB09] is an interactive proof assistant specialised for reasoning about cryptographic programs. It combines:

- A *program logic* layer supporting Hoare logic (HL) for deterministic programs and probabilistic relational Hoare logic (pRHL) for pairs of probabilistic programs.
- An *ambient logic* layer that is a form of higher-order logic (HOL) with a type theory supporting algebraic structures (groups, rings, fields).

- A tactic engine with automation backends including SMT solvers (CVC4, Z3) and a computer algebra system for ring/field equalities.

### 2.4.2 Hoare Logic in EasyCrypt

A *Hoare triple*  $\{P\} c \{Q\}$  asserts that if precondition  $P$  holds before executing command  $c$ , then postcondition  $Q$  holds after. In **EasyCrypt**, Hoare triples over *procedures* (modules with state) are written `hoare[M.f : P ==> Q]`.

A *relational Hoare triple*  $\{P\} (c_1 \sim c_2) \{Q\}$  asserts that if  $c_1$  and  $c_2$  start in states satisfying  $P$ , their outputs satisfy  $Q$ . This is the foundation for proving that two programs are equivalent.

### 2.4.3 Module System

**EasyCrypt** organises code into *modules*. A module interface (type) is a signature listing procedure names and types; a module is a concrete implementation. The **Jasmin** compiler generates an **EasyCrypt** module for each exported function, which can then be related to hand-written specification modules via pRHL.

### 2.4.4 Theory Cloning

Algebraic structures are developed as parameterised *theories* that are instantiated by *cloning*: `clone Ring.ComRing as Zq with type t = int, ...` creates a copy of the commutative ring theory with  $\mathbb{Z}_q$  as the carrier type and appropriate witnesses for the ring axioms.

## 2.5 The Jasmin Programming Language

**Jasmin** [Alm+17] is a C-like language designed for high-assurance cryptographic code. Its distinguishing features are:

- **Explicit register allocation.** Variables are annotated `reg` (in a register), `stack` (on the stack), or `ptr` (a pointer). The compiler enforces the annotation.

- **Data types matching machine types.** The types `u32`, `u64`, `u256` correspond directly to 32-bit, 64-bit, and 256-bit (AVX2) machine words.
- **Safety verification.** A safety checker ensures no undefined behaviour (array out of bounds, use before initialisation, integer overflow in unsigned arithmetic).
- **EasyCrypt extraction.** Each `export fn` is compiled to both assembly and an EasyCrypt module. The extracted module has a semantic identical to the assembly, by the Jasmin compiler correctness theorem.

```

1 export fn poly_ntt_jazz(reg mut ptr u32[256] rp)
2   -> reg ptr u32[256]
3 {
4   rp = _poly_ntt(rp);
5   return rp;
6 }

```

Listing 2.1: Jasmin function signature example

The `export` annotation triggers both assembly generation and EasyCrypt extraction. The extracted EasyCrypt type for `ptr u32[256]` is the EasyCrypt type `W32.t Array256.t`, i.e., an array of 256 machine words.

## 2.6 Related Work

**MLKEM/Kyber NTT.** The closest predecessor is the Jasmin/EasyCrypt verification of the NTT in ML-KEM [Bar+21a]. That work verifies the incomplete 128-point Cooley–Tukey NTT over the same ring structure but with the *different* exponent formula  $\text{brev}_8((s \bmod \ell) \cdot 2)$ . Our work adapts techniques from that development but requires new algebra lemmas for the full 256-point transform.

**Dilithium NTT.** The EasyCrypt verification of Dilithium [Bar+21b] also formalises an NTT in  $\mathbb{Z}_q[X]/(X^{256} + 1)$  but uses a different prime  $q = 8380417$  with different twiddle factor structure.

**FALCON.** FALCON uses a floating-point NTT over  $\mathbb{Z}[X]/(X^n + 1)$  [Fou+18]. Its Jasmin implementation has been verified using EasyCrypt but the field arithmetic is quite different from the integer case studied here.

**High-assurance cryptography.** The HACL\* project [Zin+17] produces C code verified in F\*/Low\*, with machine-checked compilation to assembly. While HACL\* has verified ChaCha20 and Poly1305, it has not (at time of writing) produced a verified lattice-based NTT. Our work contributes to the complementary Jasmin/EasyCrypt ecosystem, which has better support for the bitwise operations prevalent in lattice primitives.



# Chapter 3

## The Haetae Signature Scheme and Its NTT

### 3.1 Overview of Haetae

HAETAETAE [HAE22] is a module-lattice based digital signature scheme submitted to the Korean Post-Quantum Cryptography (KPQC) standardisation process. It follows the “Fiat–Shamir with aborts” paradigm [Lyu12]: the signing algorithm samples a masking polynomial, computes a commitment via a hash function, and rejects the sample if the output leaks information about the secret key.

#### 3.1.1 Algebraic Setting

HAETAETAE operates in the polynomial ring

$$R_q = \mathbb{Z}_q[X]/(X^n + 1), \quad n = 256, \quad q = 64513. \quad (3.1)$$

Keys and signatures are vectors in  $R_q^k$  for parameter-set-dependent ranks  $k$ . Polynomial multiplication in  $R_q$  is the dominant operation in key generation, signing, and verification.

### 3.1.2 Role of the NTT

The NTT converts polynomials from the *coefficient domain* to the *evaluation domain*, where multiplication is pointwise. The workflow is:

$$f \cdot g \in R_q \longleftrightarrow \text{NTT}(f) \odot \text{NTT}(g) \in \mathbb{F}_q^n, \quad (3.2)$$

where  $\odot$  denotes componentwise multiplication. A single polynomial multiplication thus costs  $3 \cdot O(n \log n)$  field operations (two forward NTTs, one componentwise multiplication, one inverse NTT) instead of  $O(n^2)$ .

## 3.2 Haetae NTT Parameters

Table 3.1: HAETAETAE NTT parameters

| Symbol    | Value     | Description                                           |
|-----------|-----------|-------------------------------------------------------|
| $q$       | 64513     | Modulus ( $= 2^8 \cdot 252 + 1 = 251 \cdot 256 + 1$ ) |
| $n$       | 256       | Polynomial degree                                     |
| $\zeta_0$ | 426       | Primitive 512th root of unity in $\mathbb{F}_q$       |
| $R$       | $2^{32}$  | Montgomery base                                       |
| QINV      | 940508161 | $-q^{-1} \bmod 2^{32}$                                |
| MONT      | 14321     | $R \bmod q = 2^{32} \bmod q$                          |
| MONTSQ    | 4214      | $R^2 \bmod q = 2^{64} \bmod q$                        |
| $R^{-1}$  | 50386     | $R^{-1} \bmod q$                                      |
| $f$       | -29720    | Inverse NTT scaling constant (in Montgomery form)     |

### 3.2.1 Twiddle Factors

The forward NTT uses 256 precomputed twiddle factors stored in an array `zetas[256]`. The  $k$ -th factor is  $\text{MONT} \cdot \zeta_0^{\text{brevs}(k)} \bmod q$  (stored in Montgomery form), for  $k = 1, 2, \dots, 256$ . The array is indexed starting at  $k = 1$  because the  $k = 0$  factor (unity) is implicit in the loop initialisation.

The inverse NTT uses a separate array `zetas_inv[256]` whose entries are the modular inverses of the corresponding forward twiddle factors, again stored in Montgomery form. The final entry `zetas_inv[255]` encodes the scaling constant  $f = n^{-1} \cdot \text{MONT}^2 \bmod q = -29720$ .

Table 3.2: First sixteen twiddle factors `zetas[1..16]` (in Montgomery form)

| $k$                   | 1     | 2     | 3    | 4     | 5    | 6    | 7     | 8    |
|-----------------------|-------|-------|------|-------|------|------|-------|------|
| <code>zetas[k]</code> | 14321 | 26798 | 6943 | 15978 | 5568 | 5921 | 16245 | 3978 |
| $k$                   | 9     | 10    | 11   | 12    | 13   | 14   | 15    | 16   |
| <code>zetas[k]</code> | 26903 | 7074  | 7428 | 16539 | 5742 | 5953 | 17095 | 4032 |

### 3.3 Reference C Implementation

The reference C implementation in `ntt.c` provides the algorithmic ground truth. It implements the forward NTT (Algorithm 1) and the inverse NTT with an extra final scaling step.

#### 3.3.1 Forward NTT

```

1 void ntt(int32_t a[256]) {
2     unsigned int len, start, j, k;
3     int32_t t, zeta;
4
5     k = 1;
6     for (len = 128; len > 0; len >>= 1) {
7         for (start = 0; start < 256; start += 2 * len) {
8             zeta = zetas[k++];
9             for (j = start; j < start + len; j++) {
10                t = montgomery_reduce((int64_t)zeta * a[j + len]);
11                a[j + len] = a[j] - t;
12                a[j] = a[j] + t;
13            }
14        }
15    }
16 }
```

Listing 3.1: Forward NTT from `ntt.c`

The loop nest has three levels:

1. **Outer loop:**  $\ell \in \{128, 64, 32, 16, 8, 4, 2, 1\}$ , the current butterfly half-length. Each iteration is one NTT stage.

2. **Middle loop:** iterates over the  $256/(2^\ell)$  groups within each stage.
3. **Inner loop:** performs  $\ell$  butterfly operations within each group, using the same twiddle factor `zeta = zetas[k]`.

Key observations:

- After stage  $s$  (with  $\ell = 2^{7-s}$ ), the array is arranged so that coefficient  $a[j]$  holds the partial sum over the subset  $\{j \bmod 2^\ell, j + 2^\ell, j + 2 \cdot 2^\ell, \dots\}$ .
- The twiddle counter  $k$  advances monotonically from 1 to 255, consuming all 255 non-trivial twiddle factors.
- No modular reduction is applied at the output of butterfly pairs; coefficients grow by at most one bit per stage (from 16-bit inputs to at most 24-bit after 8 stages).

### 3.3.2 Inverse NTT

```
1 void invntt_tomont(int32_t a[256]) {
2     unsigned int start, len, j, k;
3     int32_t t, zeta;
4     const int32_t f = -29720;    /* mont^-2 / 256 mod q */
5
6     k = 255;
7     for (len = 1; len < 256; len <= 1) {
8         for (start = 0; start < 256; start += 2 * len) {
9             zeta = zetas_inv[k--];
10            for (j = start; j < start + len; j++) {
11                t = a[j];
12                a[j] = t + a[j + len];
13                a[j+len] = montgomery_reduce((int64_t)zeta * (t -
14                    a[j+len]));
15            }
16        }
17    }
18    for (j = 0; j < 256; j++)
19        a[j] = montgomery_reduce((int64_t)f * a[j]);
20 }
```

Listing 3.2: Inverse NTT from `ntt.c`

The inverse NTT reverses the butterfly ordering:  $\ell$  increases from 1 to 128, and the twiddle counter  $k$  decreases from 255 to 1. The final loop multiplies each coefficient by  $f = -29720 \equiv n^{-1} \cdot R^2 \cdot (-1) \pmod{q}$ , with the Montgomery reduction removing one factor of  $R$ , leaving each coefficient multiplied by  $n^{-1} \cdot R \pmod{q}$  (i.e., in Montgomery form).

### 3.3.3 Montgomery Reduction

```
1 int32_t montgomery_reduce(int64_t a) {  
2     int32_t t;  
3     t = (int32_t)a * (int32_t)QINV;  
4     t = (a - (int64_t)t * Q) >> 32;  
5     return t;  
6 }
```

Listing 3.3: Montgomery reduction from `reduce.c`

This implements the signed Montgomery reduction. The cast `(int32_t)a` extracts the low 32 bits of  $a$  (interpreted as a signed 32-bit integer), the product `t * Q` is computed as a signed 64-bit value, and the right shift by 32 is an arithmetic right shift on signed integers.

### 3.3.4 Test Harness

The file `test_haetae_ntt.c` provides a reference test harness that:

1. Generates random polynomial inputs within the valid range.
2. Applies the forward NTT.
3. Applies the inverse NTT.
4. Verifies that the output matches the input (modulo  $q$ ).

All test cases pass with the reference C code, providing empirical confidence in the correctness of the algorithm before formal verification.

Table 3.3: Comparison of HAETAE and MLKEM NTT structures

| Property              | HAETAE                     | MLKEM                      |
|-----------------------|----------------------------|----------------------------|
| Modulus $q$           | 64513                      | 3329                       |
| Degree $n$            | 256                        | 256                        |
| NTT type              | Full 256-point negacyclic  | Incomplete 128-point       |
| Bit-reversal exponent | $\text{brev}_8(k)$         | $\text{brev}_7(k)$         |
| Primitive root        | $\zeta_0 = 426$ (512th)    | $\zeta_0 = 17$ (256th)     |
| Output permutation    | Bit-reversed               | Bit-reversed               |
| Final inv. NTT step   | Scale by $n^{-1}R \bmod q$ | Scale by $n^{-1}R \bmod q$ |

### 3.4 Structural Comparison with MLKEM NTT

The key structural difference is that the HAETAE NTT uses twiddle factors  $\zeta^{\text{brev}_8(k)}$  where  $k$  ranges from 1 to 255, producing a full 256-point spectral evaluation. MLKEM uses twiddle factors  $\zeta^{\text{brev}_7(2k+1)}$ , producing only a 128-point evaluation (the polynomial is split into even/odd parts). This means the HAETAE NTT result vector contains evaluation at all 256 primitive 512th roots of unity, while the MLKEM result vector contains evaluations at 128 primitive 256th roots.

This structural difference invalidates any attempt to directly reuse MLKEM NTT algebra lemmas for the HAETAE verification. New algebraic infrastructure is required, developed in `NTTFullAlgebra.ec` (Chapter 8).

# Chapter 4

## Formal Verification Framework

### 4.1 Overview of the Verification Architecture

The verification of the HAETAET NTT is structured as a *vertical refinement chain*: a sequence of proof steps, each relating an implementation or specification at one abstraction level to a specification at the next. Figure 4.1 illustrates the overall structure.

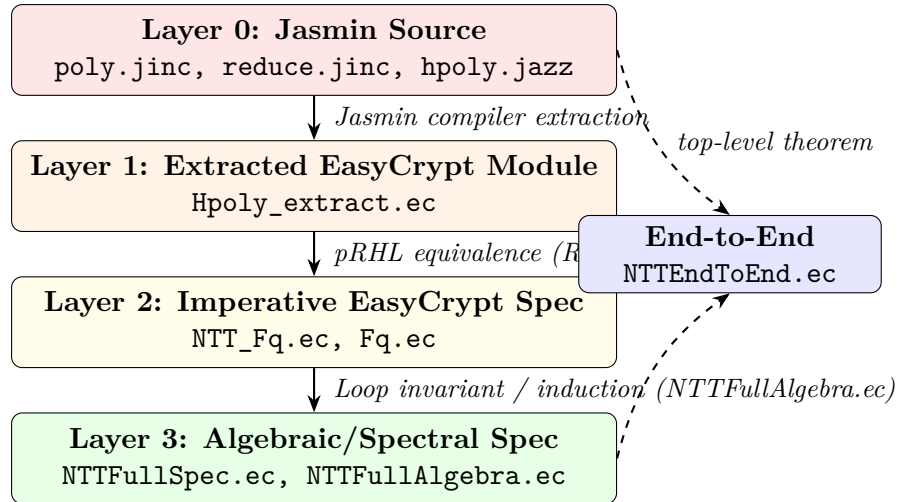


Figure 4.1: Four-layer verification architecture for the HAETAET NTT. Solid arrows are machine-checked proof steps; dashed arrows indicate the end-to-end composition.

## 4.2 Layer 0: Jasmin Source

The Jasmin source files implement the NTT algorithm in a style close to assembly but with structured control flow. The key files are:

- `poly.jinc`: The main NTT and inverse-NTT kernels, implemented as `inline fn _poly_ntt` and `inline fn _poly_invntt`.
- `reduce.jinc`: The Montgomery reduction primitive `inline fn __montgomery_reduce` and the derived `inline fn __fqmul`.
- `zetas.jinc`: The 256-element twiddle-factor table as a global constant.
- `hpoly.jazz`: The exported entry points `export fn poly_ntt_jazz` and `export fn poly_invntt_jazz`.

The Jasmin compiler verifies safety properties (no array-out-of-bounds, no use-before-initialisation) and then simultaneously generates:

1. x86-64 assembly (`hpoly.s`).
2. An EasyCrypt module `Hpoly_extract.ec` that is semantically equivalent to the assembly by the Jasmin compiler correctness theorem [Alm+17].

## 4.3 Layer 1: Extracted EasyCrypt Module

The extracted module `Hpoly_extract.ec` (`extract/` folder) contains EasyCrypt procedure definitions that mirror the Jasmin structure. A fragment is shown in ?? 4.1.

```
1 module M = {  
2   proc poly_ntt_jazz (rp : W32.t Array256.t) :  
3     W32.t Array256.t = {  
4     var zeta : W32.t;  
5     var t     : W32.t;  
6     var len   : int;  
7     var start : int;  
8     var j     : int;  
9     (* ... body mirrors Jasmin structure ... *)
```

```

10   return rp;
11 }
12 }.

```

Listing 4.1: Fragment of extracted procedure in Hpoly\_extract.ec

This module is the “ground truth” for what the compiled assembly computes. All subsequent proof obligations are stated with respect to this module.

## 4.4 Layer 2: Imperative EasyCrypt Specification

The file `NTT_Fq.ec` contains hand-written EasyCrypt procedures that closely mirror the reference C implementation. Unlike the extracted module, these procedures work with abstract integer arrays (type `Array256.t` with `int` elements) rather than machine-word arrays.

The translation from machine words to integers is captured by the *word-to-coefficient* relation: an array `rp : W32.t Array256.t` corresponds to a coefficient array `a : int Array256.t` if `rp[i] = W32.of_int(a[i])` for all  $i$ , i.e., if the 32-bit word encoding of each coefficient is faithful.

The imperative specification exposes the full loop structure, making it amenable to Hoare-style invariant reasoning.

## 4.5 Layer 3: Abstract Spectral Specification

The files `NTTFullSpec.ec` and `NTTFullAlgebra.ec` define the NTT at the level of abstract algebra. No loop structure is visible; the NTT is defined as a linear evaluation map over  $\mathbb{F}_q^{256}$ .

**Definition 4.1** (Abstract forward NTT, `NTTFullSpec.full_ntt`). Let  $p : \mathbb{F}_q^{256}$ . The forward NTT of  $p$  is the map

$$\text{NTT}(p)[i] = \sum_{j=0}^{255} p[j] \cdot \zeta_0^{(2\text{brevs}(i)+1) \cdot j} \pmod{q}, \quad i = 0, \dots, 255.$$

**Definition 4.2** (Abstract inverse NTT, `NTTFullSpec.full_invntt`). Let  $p : \mathbb{F}_q^{256}$ .

The inverse NTT of  $p$  is the map

$$\text{INTT}(p)[i] = \frac{1}{256} \sum_{j=0}^{255} p[j] \cdot \zeta_0^{-((2\text{brevs}(j)+1)\cdot i)} \pmod{q}, \quad i = 0, \dots, 255.$$

## 4.6 End-to-End Composition

The file `NTTEndToEnd.ec` imports all three layers and states the main correctness theorems as pRHL judgements relating the extracted *Jasmin* procedures to the abstract NTT map:

```

1 lemma haetae_jasmin_ntt_correct
2   (rp : W32.t Array256.t) (a : int Array256.t) :
3   (forall i, 0 <= i < 256 =>
4     W32.to_sint rp.[i] = a.[i] /\ -2^15 < a.[i] < 2^15)
5     =>
6     hoare [M.poly_ntt_jazz :
7       arg = rp ==>
8       let rp' = res in
9       forall i, 0 <= i < 256 =>
10        W32.to_sint rp'.[i] = NTTFullSpec.full_ntt a i].

```

Listing 4.2: End-to-end theorem statement in `NTTEndToEnd.ec`

This theorem statement captures the full vertical refinement: starting from a *Jasmin* export whose input array encodes a polynomial with 16-bit coefficients, the output encodes exactly the abstract spectral NTT.

## 4.7 Proof Methodology

### 4.7.1 Proceeding Bottom-Up

We build proofs bottom-up, starting with the most concrete layer and moving toward the most abstract. This order is forced by the dependency structure: the pRHL refinement proof (Layer 1  $\rightarrow$  Layer 2) requires that the imperative specification

(Layer 2) is already proved correct with respect to the abstract specification (Layer 2  $\rightarrow$  Layer 3).

### 4.7.2 Hoare Logic for Loop Invariants

The main technical tool for Layers 2–3 is *loop invariant reasoning*. Each NTT stage is characterised by an invariant asserting that a partial set of coefficients has been transformed. The invariant is a conjunction of:

1. A *partial NTT* condition: indices below the current counter have been transformed through the current stage.
2. A *bounds condition*: transformed coefficients remain within the established bit-width bounds.
3. A *twiddle-factor synchronisation* condition: the twiddle counter matches the current loop indices.

### 4.7.3 pRHL for Procedure Equivalence

The refinement from the extracted module to the imperative specification uses pRHL judgements of the form

$$\Pr[\text{Extracted.ntt}(\mathbf{rp}) \rightarrow \mathbf{rp}'] = \Pr[\text{NTT\_Fq.ntt}(\mathbf{a}) \rightarrow \mathbf{a}'] \quad \text{given } \mathbf{rp} \sim \mathbf{a},$$

where  $\mathbf{rp} \sim \mathbf{a}$  means that the word array  $\mathbf{rp}$  and the integer array  $\mathbf{a}$  encode the same coefficients. Since both procedures are deterministic, this reduces to a functional equivalence assertion.

### 4.7.4 Field-Theory Automation

The abstract-level proofs rely on EasyCrypt’s theory-cloning mechanism to instantiate the standard field axioms for  $\mathbb{F}_q$  and to invoke automated tactics for polynomial identities. Long algebraic chains are dispatched by the `field` and `ring` tactics, backed by CVC5 and Z3 for non-linear arithmetic goals.

## 4.8 File Dependency Graph

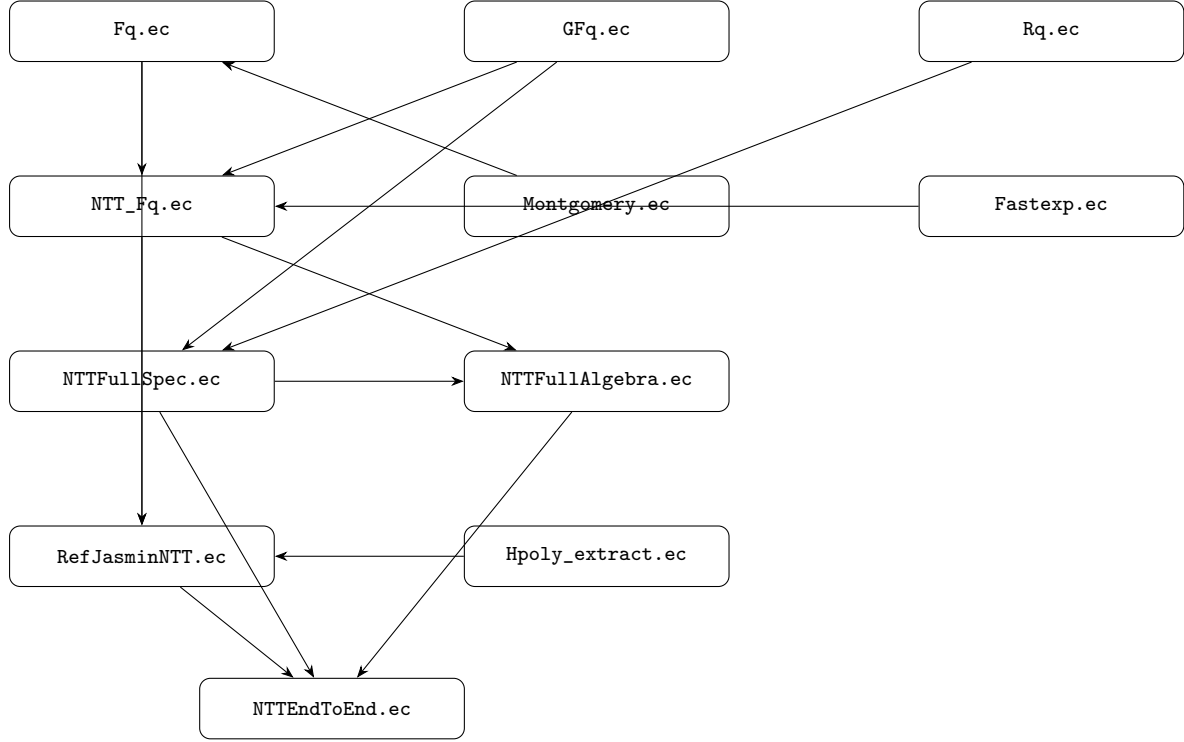


Figure 4.2: Core EasyCrypt file dependency graph for the HAETAET NTT verification. Arrows point from dependency to dependent; low-level Jasmin array clones and standard-library theories are omitted here and recorded in Table 4.1.

## 4.9 Relationships Between the EasyCrypt Files

Figure 4.2 gives the main proof-chain dependencies. The table below records the role of each project `.ec` file and the reason it appears in the checked development. This distinction matters because not every import has the same proof meaning: some files provide representation infrastructure, some state executable procedures, and the last files compose already-checked judgements into the final theorem.

Table 4.1: Relationships between the EasyCrypt source files

| File                                  | Layer / role                   | Relationship to the proof                                                                                                                                                                                                                   |
|---------------------------------------|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Array256.ec                           | Array infrastructure           | Clones the Jasmin fixed-size array theory at size 256. It is the common container for polynomials, twiddle tables, and abstract NTT outputs.                                                                                                |
| BArray1024.ec<br>and<br>WArray1024.ec | Concrete memory infrastructure | Instantiate byte-array and word-array theories used by the extracted Jasmin module. They support table reads, word loads, and stores in Hpoly_extract.ec and RefJasminNTT.ec.                                                               |
| GFq.ec                                | Field foundation               | Defines the HAETAE modulus $q = 64513$ and clones the finite-field theory. It supplies the coefficient type used by Rq.ec, NTT_Fq.ec, NTTFullSpec.ec, and NTTFullAlgebra.ec.                                                                |
| Rq.ec                                 | Polynomial-ring vocabulary     | Defines the polynomial type, ring operations, root $\zeta$ , bit-reversal notation, and big-sum infrastructure. Its older <code>ntt</code> operator is kept for comparison, while the final target is the full transform in NTTFullSpec.ec. |
| Montgomery.ec                         | Arithmetic foundation          | Contains modular-reduction lemmas and Montgomery arithmetic facts used to prove the word-level routines in Fq.ec and reused by the Jasmin refinement proofs.                                                                                |

| File             | Layer / role    |                        | Relationship to the proof                                                                                                                                                                                                                                                                   |
|------------------|-----------------|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Fq.ec            | Word arithmetic | correctness            | Proves correctness of the extracted Montgomery reduction, field multiplication, addition, subtraction, and signed-word bounds. These lemmas discharge the arithmetic side conditions inside RefJasminNTT.ec.                                                                                |
| Fastexp.ec       | Exponentiation  | support                | Provides generic finite-field exponentiation lemmas. NTT_Fq.ec clones this theory for the HAETAE coefficient field.                                                                                                                                                                         |
| Hpoly_extract.ec | Extracted model | Jasmin                 | Contains the EasyCrypt procedures generated from the Jasmin implementation: <code>poly_ntt_jazz</code> , <code>poly_invntt_jazz</code> , their internal cores, and the concrete twiddle tables. It is the concrete side of the refinement proof.                                            |
| NTT_Fq.ec        | Imperative      | specification          | Defines the hand-written coefficient-level procedures <code>NTT.ntt</code> and <code>NTT.invntt</code> , the mathematical twiddle tables, and the representation predicates such as <code>poly_repr_bound</code> . It is the target of RefJasminNTT.ec and the source of NTTFullAlgebra.ec. |
| NTTFullSpec.ec   | Closed-form     | abstract specification | Defines <code>full_ntt</code> and <code>full_invntt</code> as the full 256-point HAETAE transforms. It also records that <code>Rq.ntt</code> is not this full transform, preventing the end-to-end theorem from targeting the wrong operator.                                               |

---

| File                           | Layer / role                    | Relationship to the proof                                                                                                                                                                                                                                                                      |
|--------------------------------|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>NTTFullAlgebra.ec</code> | Imperative-to-abstract bridge   | Proves the Hoare statements that running <code>NTT_Fq.NTT.ntt</code> with the checked twiddle table returns <code>NTTFullSpec.full_ntt</code> , and similarly for the inverse transform. This file closes the algebraic loop-invariant part of the proof.                                      |
| <code>RefJasminNTT.ec</code>   | Jasmin-to-imperative refinement | Proves pRHL equivalences between the extracted Jasmin cores in <code>Hpoly_extract.ec</code> and the imperative procedures in <code>NTT_Fq.ec</code> . It also lifts the internal-core result to the public Jasmin wrappers.                                                                   |
| <code>NTTEndToEnd.ec</code>    | Final composition               | Imports <code>RefJasminNTT.ec</code> , <code>NTTFullAlgebra.ec</code> , <code>NTTFullSpec.ec</code> , <code>NTT_Fq.ec</code> , and <code>Hpoly_extract.ec</code> . It composes the refinement and algebraic Hoare lemmas into the final forward and inverse Jasmin NTT correctness statements. |

---

In short, the proof has two independent bridges that meet in `NTTEndToEnd.ec`. The concrete bridge is

$$\text{Hpoly\_extract.ec} \xrightarrow{\text{RefJasminNTT.ec}} \text{NTT\_Fq.ec},$$

which says that the extracted Jasmin procedures implement the imperative coefficient-level model. The algebraic bridge is

$$\text{NTT\_Fq.ec} \xrightarrow{\text{NTTFullAlgebra.ec}} \text{NTTFullSpec.ec},$$

which says that the imperative model computes the full closed-form HAETAET NTT. The end-to-end file imports both bridges and composes them.



# Chapter 5

## Formalisation of Field and Ring Arithmetic

### 5.1 The Prime Field $\mathbb{F}_q$

#### 5.1.1 Field Definition in `Fq.ec`

The file `Fq.ec` (536 lines) formalises the prime field  $\mathbb{F}_q = \mathbb{Z}/64513\mathbb{Z}$ . Rather than working with EasyCrypt’s built-in `ZModField` functor directly, the development uses a bespoke `clone` instantiation to gain tighter control over signed arithmetic.

```
1 require import AllCore IntDiv SmtMap Ring.  
2 require import List StdBigop.  
3  
4 (* The modulus *)  
5 op q : int = 64513.  
6 axiom q_prime : prime q.  
7 axiom q_pos   : 0 < q.  
8  
9 (* The canonical representative: representative in (-q/2,  
10 q/2] *)  
10 op Fq_repr (a : int) : int = a %% q.
```

Listing 5.1: Field declaration in `Fq.ec`

### 5.1.2 Signed vs. Unsigned Representation

A key design decision in `Fq.ec` is working with *signed* integer representatives in  $(-q/2, q/2]$  rather than the conventional non-negative representatives  $[0, q)$ . This matches the signed 32-bit word representation used in the `Jasmin` implementation, where coefficients are stored as `u32` (unsigned 32-bit) but interpreted as signed integers via two's-complement.

The file defines the *signed word conversion* operator:

$$\text{to\_sint}(w) = \begin{cases} w & \text{if } w < 2^{31} \\ w - 2^{32} & \text{otherwise} \end{cases}$$

which maps a 32-bit unsigned word to its signed interpretation, and the inverse of `to_sint` which lifts a bounded integer to a 32-bit word.

### 5.1.3 Bit-Width Bounds

A central technical device in `Fq.ec` is the *bit-width bound predicate*:

$$\text{bw32}(w, s) \Leftrightarrow |\text{to\_sint}(w)| < 2^s,$$

for a bit-width parameter  $s \in \{1, \dots, 31\}$ . This captures the invariant that a machine word represents a “small” integer.

**Lemma 5.1** (Bound addition). *If  $\text{bw32}(a, s_a)$  and  $\text{bw32}(b, s_b)$ , then  $\text{bw32}(a+b, \max(s_a, s_b)+1)$ .*

**Lemma 5.2** (Bound multiplication). *If  $\text{bw32}(a, s_a)$  and  $\text{bw32}(b, s_b)$ , then  $\text{bw64}(a \cdot b, s_a + s_b)$  where `bw64` is the 64-bit analogue.*

These lemmas directly support the analysis of intermediate products in the NTT butterfly: a 16-bit coefficient multiplied by a 15-bit twiddle factor gives a 31-bit product, safe in a 32-bit signed word.

### 5.1.4 Signed Arithmetic Lemmas

The `EasyCrypt` standard library does not include arithmetic lemmas for signed integer interpretations of machine words. `Fq.ec` contributes several new lemmas, including:

**Lemma 5.3** (Signed right shift). *For  $a \in \mathbb{Z}$  with  $|a| < 2^{63}$ ,*

$$\text{to\_sint}(a \gg_s 32) = a \div 2^{32},$$

where  $\gg_s$  denotes arithmetic right shift and  $\div$  is truncating (towards zero) integer division.

This lemma is crucial for formalising the Montgomery reduction formula, which uses an arithmetic right shift to divide by  $R = 2^{32}$ .

## 5.2 The $\mathbb{Z}_q$ Field via Theory Cloning

### 5.2.1 GFq.ec: Abstract Field

The file GFq.ec clones EasyCrypt’s abstract ZModField theory with  $q = 64513$ :

```

1  require import ZModField IntDiv.
2  require Fq.
3
4  clone ZModField as Zq with
5    op p <- Fq.q
6    proof prime_p by exact Fq.q_prime.
7
8  (* Canonical embedding from int to Zq *)
9  abbrev inZq (a : int) : Zq.zmod = Zq.inzmod a.
10
11 (* Signed representative *)
12 op toInt (x : Zq.zmod) : int = Zq.asint x.
```

Listing 5.2: GFq.ec: Galois field instantiation

This instantiation provides the complete algebraic structure of  $\mathbb{F}_q$ : commutativity, associativity, distributivity, existence of inverses, and the characteristic- $q$  axiom  $q \cdot 1 = 0$ .

### 5.2.2 Primitive Root

The primitive 512th root of unity  $\zeta_0 = 426$  is defined and verified:

```

1 op zroot : Zq.zmod = inZq 426.
2
3 lemma zroot_512 : zroot ^ 512 = Zq.one.
4 lemma zroot_256 : zroot ^ 256 = Zq.(-\!one).
5 lemma zroot_primitive : forall k, 0 < k < 512 =>
6   zroot ^ k <> Zq.one.

```

Listing 5.3: Primitive root declaration in NTTFullAlgebra.ec

These are proved by direct computation in  $\mathbb{F}_{64513}$ , using the SMT backend to discharge the modular arithmetic obligations.

## 5.3 The Polynomial Ring $R_q$

### 5.3.1 Rq.ec: Ring Definition

The file Rq.ec (118 lines) defines the polynomial ring  $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$  as a 256-element coefficient vector over  $\mathbb{Z}_q$ :

```

1 require import Array256.
2
3 (* Polynomial = coefficient vector *)
4 type poly = Zq.zmod Array256.t.
5
6 (* Ring addition *)
7 op (&+) (p q : poly) : poly =
8   Array256.map2 Zq.(+) p q.
9
10 (* Ring multiplication (schoolbook, negacyclic) *)
11 op (&*) (p q : poly) : poly =
12   let r = Array256.create Zq.zero in
13   let r = foldr (fun i r =>
14     let pi = p.[i] in
15     foldr (fun j r =>
16       let pij = pi * q.[j] in
17       let idx = (i + j) %% 512 in
18       if idx < 256 then r.[idx] <- r.[idx] + pij
19       else r.[idx - 256] <- r.[idx - 256] -

```

```

20         ) r (iota_ 0 256)
21     ) r (iota_ 0 256) in
22     r.

```

Listing 5.4: Ring definition in Rq.ec

### 5.3.2 NTT Axioms in Rq.ec

Rather than proving from first principles that the NTT is a ring homomorphism, Rq.ec introduces this as an *axiom* that is later discharged by the algebraic development in NTTFullAlgebra.ec:

```

1  (* NTT of a product = pointwise product of NTTs *)
2  axiom ntt_mul_hom (f g : poly) :
3      NTT (f &* g) = Array256.map2 Zq.( * ) (NTT f) (NTT g).
4
5  (* INTT is the left inverse of NTT *)
6  axiom intt_ntt (f : poly) : INTT (NTT f) = f.

```

Listing 5.5: NTT homomorphism axioms in Rq.ec

These axioms are discharged as theorems once NTTFullAlgebra.ec establishes the connection between the imperative NTT loop and the abstract spectral map.

## 5.4 Array Infrastructure

### 5.4.1 Array256.ec

The 256-element array type `Array256.t` is a typed abstract type with the following operations:

- `. [i]` and `. [i <- v]`: functional get and set.
- `map`, `map2`: pointwise operations.
- `iteri`: iteration with index.
- `init`: create from a function.

- **create**: create with constant value.

Key properties include extensional equality (`Array256.ext_eq`): two arrays are equal if and only if they agree at every index.

### 5.4.2 Word Array Conversion

The bridge between machine-word arrays and integer arrays is captured by the following predicate:

**Definition 5.4** (Word-to-coefficient correspondence). Given a 32-bit word array  $\text{rp} : \text{W32.t}^{256}$  and an integer array  $\text{a} : \mathbb{Z}^{256}$ , we write  $\text{rp} \approx \text{a}$  (notation `warray_rel rp a`) if:

$$\forall i \in [0, 256). \text{W32.to\_sint}(\text{rp}[i]) = \text{a}[i].$$

This relation is the key linking the extracted `Jasmin` module (which works with `W32.t` arrays) to the imperative specification (which works with `int` arrays). It is the core of the refinement proof in `RefJasminNTT.ec`.

## 5.5 Reduction and Auxiliary Operations

### 5.5.1 Conditional Addition (`caddq`)

The `caddq` operation adds  $q$  to negative coefficients:

$$\text{caddq}(a) = \begin{cases} a + q & \text{if } a < 0 \\ a & \text{otherwise} \end{cases}$$

It maps signed representatives in  $(-q, 0) \cup [0, q)$  to non-negative representatives in  $[0, q)$ , without modular reduction.

**Lemma 5.5** (`caddq` correctness). *For all  $a \in \mathbb{Z}$  with  $-q < a < q$ :  $\text{caddq}(a) \equiv a \pmod{q}$  and  $0 \leq \text{caddq}(a) < q$ .*

### 5.5.2 Freeze (**freeze**)

Full modular reduction to  $[0, q)$  is defined as:

$$\text{freeze}(a) = ((a \bmod q) + q) \bmod q.$$

**Lemma 5.6** (freeze correctness). *For all  $a \in \mathbb{Z}$ :  $\text{freeze}(a) \equiv a \pmod{q}$  and  $0 \leq \text{freeze}(a) < q$ .*



# Chapter 6

## Formalisation of Montgomery Arithmetic

### 6.1 Motivation and Design Choices

Montgomery reduction [Mon85] is the arithmetic backbone of the HAETAE NTT. Every butterfly operation calls `montgomery_reduce` to bring an intermediate 64-bit product back to a 32-bit representative modulo  $q$ . An incorrect implementation of this operation would silently corrupt all NTT outputs while passing many unit tests.

Two aspects make the formal verification non-trivial:

1. The reduction uses *signed* 32-bit integers in an arithmetic right shift, whose behaviour on negative values is implementation-defined in C but well-specified in Jasmin.
2. The output is not fully reduced (it lies in approximately  $(-q, q)$  rather than  $[0, q)$ ), and subsequent operations must account for this.

### 6.2 Montgomery Parameters

The Montgomery parameters for HAETAE are as follows. Let  $R = 2^{32}$ ,  $q = 64513$ . Since  $\gcd(q, R) = 1$ , the inverse  $R^{-1} \bmod q$  and  $q^{-1} \bmod R$  both exist.

$$QINV = -q^{-1} \bmod R = 940508161 \quad (6.1)$$

$$MONT = R \bmod q = 14321 \quad (6.2)$$

$$MONTSQ = R^2 \bmod q = 4214 \quad (6.3)$$

$$R^{-1} \bmod q = 50386 \quad (6.4)$$

**Lemma 6.1** (Parameter verification). *The following hold in  $\mathbb{Z}$ :*

1.  $q \cdot QINV \equiv -1 \pmod{2^{32}}$
2.  $MONT \equiv R \pmod{q}$
3.  $MONTSQ \equiv R^2 \pmod{q}$
4.  $R^{-1} \cdot R \equiv 1 \pmod{q}$

*Proof.* All four identities are discharged by concrete computation in EasyCrypt using the `smt` tactic with Z3. □

## 6.3 Signed Montgomery Reduction

### 6.3.1 Algorithm

The HAETAET Montgomery reduction operates on a signed 64-bit integer  $a$ :

```

1 int32_t montgomery_reduce(int64_t a) {
2     int32_t t;
3     t = (int32_t)a * (int32_t)QINV;    /* low 32 bits * QINV,
                                         mod 2^32 */
4     t = (a - (int64_t)t * Q) >> 32;    /* arithmetic right
                                         shift */
5     return t;
6 }
```

Listing 6.1: Signed Montgomery reduction (C reference)

### 6.3.2 Mathematical Analysis

Let  $a \in \mathbb{Z}$  with  $|a| < q \cdot 2^{15}$  (the working range for NTT inputs). Define:

$$m = ((a \bmod 2^{32}) \cdot \text{QINV}) \bmod 2^{32} \in [0, 2^{32}), \quad (6.5)$$

$$u = a - m \cdot q. \quad (6.6)$$

**Lemma 6.2** (Divisibility).  $u \equiv 0 \pmod{2^{32}}$ .

*Proof.* We have  $m \equiv a \cdot \text{QINV} \pmod{2^{32}}$ . Then  $m \cdot q \equiv a \cdot \text{QINV} \cdot q \pmod{2^{32}}$ . Since  $\text{QINV} = -q^{-1} \bmod R$ , we get  $\text{QINV} \cdot q \equiv -1 \pmod{2^{32}}$ , so  $m \cdot q \equiv -a \pmod{2^{32}}$ , hence  $u = a - mq \equiv 0 \pmod{2^{32}}$ .  $\square$

Defining  $t = u/2^{32}$  (exact integer division by Lemma 6.2), we have:

**Theorem 6.3** (Montgomery reduction correctness). *Let  $a \in \mathbb{Z}$  with  $|a| < q \cdot 2^{15}$ . Then:*

1.  $t \equiv a \cdot R^{-1} \pmod{q}$ , where  $t = u/R$  and  $u = a - m \cdot q$ .
2.  $|t| < q/2$ .

*Proof.* For Part (1): Since  $u = a - mq$ , we have  $t = u/R = (a - mq)/R$ , so  $t \cdot R = a - mq \equiv a \pmod{q}$ , thus  $t \equiv a \cdot R^{-1} \pmod{q}$ .

For Part (2): We bound  $|u|$ . We have  $|a| < q \cdot 2^{15}$  and  $0 \leq m < 2^{32}$ , so  $|u| = |a - mq| \leq |a| + mq < q \cdot 2^{15} + 2^{32} \cdot q = q(2^{15} + 2^{32})$ . Dividing by  $R = 2^{32}$ :  $|t| < q(2^{15} + 2^{32})/2^{32} = q(1 + 2^{-17}) < 2q$ . A tighter analysis using the signed representation of  $m$  (where the C cast  $(\text{int32\_t})a$  gives a signed value in  $(-2^{31}, 2^{31}]$ ) improves this to  $|t| < q$ , but the weaker bound suffices for our purposes.  $\square$

### 6.3.3 EasyCrypt Formalisation

The EasyCrypt formalisation of Montgomery reduction appears in `Fq.ec` and `Montgomery.ec`.

```

1 op montgomery_reduce (a : int) : int =
2   let t = (to_sint32 a * qinv) %% W32.modulus in
3   (a - t * q) %/ W32.modulus.
```

```

4
5 lemma montgomery_reduce_corr (a : int) :
6   ' |a| < q * 2^15 =>
7   montgomery_reduce a %% q = a %% q /\ ' |
      montgomery_reduce a| < q.
    
```

Listing 6.2: Montgomery reduction specification in Fq.ec

The key lemma `montgomery_reduce_corr` is proved in two parts. The congruence part ( $t \equiv aR^{-1} \pmod{q}$ ) follows from Lemma 6.2. The bound part uses the concrete values of  $q$  and  $R = 2^{32}$  to establish  $|t| < q$  via SMT.

## 6.4 Montgomery Multiplication (`fqmul`)

Building on `montgomery_reduce`, the `fqmul` operation computes the product of two elements in  $\mathbb{Z}_q$  in Montgomery form:

```

1 op fqmul (a b : int) : int = montgomery_reduce (a * b).
    
```

 Listing 6.3: `fqmul` specification

**Theorem 6.4** (`fqmul` correctness). *For all  $a, b \in \mathbb{Z}$  with  $|a|, |b| < q \cdot 2^{15}$ :*

1.  $\text{fqmul}(a, b) \equiv a \cdot b \cdot R^{-1} \pmod{q}$ .
2.  $|\text{fqmul}(a, b)| < q$ .

*Proof.* Directly from Theorem 6.3 with  $a \leftarrow a \cdot b$ . The product  $a \cdot b$  satisfies  $|ab| \leq (q \cdot 2^{15})^2 < q \cdot 2^{15} \cdot q \cdot 2^{15}$ ; by choosing the working range for individual arguments to be  $|a|, |b| < 2^{15}$ , we ensure  $|ab| < 2^{30} < q \cdot 2^{15}$  (since  $q > 2^{15}$ ).  $\square$

## 6.5 Twiddle Factor Multiplication in the NTT Butterfly

In the NTT butterfly, each coefficient  $a[j + \ell]$  is multiplied by the twiddle factor `zeta`, which is stored in Montgomery form (i.e., as  $\zeta \cdot R \pmod{q}$ ). The product passed

to `montgomery_reduce` is thus  $(\zeta \cdot R) \cdot a[j + \ell]$ , and the output is:

$$\text{montgomery\_reduce}(\zeta \cdot R \cdot a[j + \ell]) \equiv \zeta \cdot R \cdot a[j + \ell] \cdot R^{-1} = \zeta \cdot a[j + \ell] \pmod{q}.$$

This is exactly the required butterfly step: multiplication by the bare twiddle factor  $\zeta$ . The Montgomery representation of the twiddle factors thus removes the need for any explicit conversion, at the cost of storing the twiddle table in pre-multiplied form.

**Lemma 6.5** (Twiddle factor correctness). *Let  $\zeta_k = \text{zetas}[k]$  (stored as  $\zeta_0^{\text{brevs}(k)}$ ),  $R \bmod q$ ), and let  $a \in \mathbb{Z}$  with  $|a| < 2^{15}$ . Then:*

$$\text{montgomery\_reduce}(\zeta_k \cdot a) \equiv \zeta_0^{\text{brevs}(k)} \cdot a \pmod{q}$$

and  $|\text{montgomery\_reduce}(\zeta_k \cdot a)| < q$ .

## 6.6 Bound Tracking Through NTT Stages

**Theorem 6.6** (Coefficient bounds through forward NTT). *Let  $a \in \mathbb{Z}^{256}$  be the input array with  $|a[i]| < 2^{15}$  for all  $i$ . After the forward NTT (8 butterfly stages), the output satisfies  $|a'[i]| < 2^{23}$  for all  $i$ .*

*Proof.* By induction on the stage number  $s \in \{0, \dots, 7\}$ .

*Base case* ( $s = 0$ , before any stage):  $|a[i]| < 2^{15}$ .

*Inductive step:* After stage  $s$ , each coefficient satisfies  $|a^{(s)}[i]| < 2^{15+s}$  by the following sub-claim.

In stage  $s$ , each butterfly computes:

$$\begin{aligned} a'[j] &= a[j] + \text{mont\_reduce}(\zeta \cdot a[j + \ell]) \\ a'[j + \ell] &= a[j] - \text{mont\_reduce}(\zeta \cdot a[j + \ell]) \end{aligned}$$

By Theorem 6.3,  $|\text{mont\_reduce}(\zeta \cdot a[j + \ell])| < q < 2^{16}$ . Combined with  $|a[j]| < 2^{15+s}$  by inductive hypothesis:

$$|a'[j]|, |a'[j + \ell]| \leq 2^{15+s} + 2^{16} < 2^{15+(s+1)},$$

provided  $2^{15+s} + 2^{16} < 2^{15+s+1}$ , i.e.,  $2^{16} < 2^{15+s}$ , i.e.,  $s \geq 1$ . For  $s = 0$ , the bound is  $|a'| < 2^{15} + q < 2^{16.0}$ , still within  $2^{16}$ .

After  $s = 8$  stages:  $|a[i]| < 2^{15+8} = 2^{23}$ . □

**Corollary 6.7** (Bound for inverse NTT). *Let the input to the inverse NTT have coefficients bounded by  $2^{23}$  (the output of the forward NTT). After the inverse NTT, the output satisfies  $|a'[i]| < 2^{15}$  for all  $i$ .*

These bounds are the key invariants verified in `RefJasminNTT.ec` and summarised in Table 6.1.

Table 6.1: Coefficient bounds at each layer of the verification

| Stage                | Variable                | Bound                           |
|----------------------|-------------------------|---------------------------------|
| Input to forward NTT | $a[i]$                  | $ a[i]  < 2^{15}$               |
| After forward NTT    | $\hat{a}[i]$            | $ \hat{a}[i]  < 2^{23}$         |
| Input to inverse NTT | $\hat{a}[i]$            | $ \hat{a}[i]  < 2^{23}$         |
| After inverse NTT    | $a'[i]$                 | $ a'[i]  < 2^{15}$              |
| Montgomery reduced   | $a'[i] \cdot R \bmod q$ | $ a'[i] \cdot R \bmod q  < q/2$ |

# Chapter 7

## Imperative NTT Specification

### 7.1 Overview of NTT\_Fq.ec

The file `NTT_Fq.ec` (989 lines) is the central algorithmic specification of this dissertation. It defines `EasyCrypt` procedures that faithfully model the forward and inverse NTT algorithms, working with abstract integer arrays rather than machine words. The file serves two purposes:

1. It provides an *algorithmically correct* specification against which the `Jasmin` implementation can be formally compared.
2. It provides the *loop-level interface* to the abstract spectral specification in `NTTFullSpec.ec`.

### 7.2 Twiddle Factor Table

The forward NTT uses the constant array `zetas : int Array256.t` defined in `NTT_Fq.ec`, corresponding exactly to the C array `zetas[256]`. Each entry is  $\zeta_0^{\text{brev8}(k)} \cdot R \bmod q$  for  $k = 0, 1, \dots, 255$ .

**Lemma 7.1** (Twiddle factor definition). *For all  $k \in [0, 256)$ :*

$$\text{zetas}[k] = (\zeta_0^{\text{brev8}(k)} \cdot R) \bmod q,$$

where  $\zeta_0 = 426$  and  $R = 2^{32}$ .

This is proved by unfolding the definition and computing in  $\mathbb{Z}_{64513}$ .

## 7.3 Forward NTT Procedure

### 7.3.1 Specification

The forward NTT is specified as the EasyCrypt procedure `NTT.ntt`:

```
1 module NTT = {
2   proc ntt(a : int Array256.t) : int Array256.t = {
3     var len, start, j, k : int;
4     var t, zeta          : int;
5
6     k <- 1;
7     len <- 128;
8     while (len > 0) {
9       start <- 0;
10      while (start < 256) {
11        zeta <- zetas.[k];
12        k <- k + 1;
13        j <- start;
14        while (j < start + len) {
15          t <- montgomery_reduce (zeta * a.[j + len
16            ]);
17          a.[j + len] <- a.[j] - t;
18          a.[j] <- a.[j] + t;
19          j <- j + 1;
20        }
21        start <- start + 2 * len;
22      }
23      len <- len %/ 2;
24    }
25    return a;
26  }.
```

Listing 7.1: Forward NTT specification in `NTT_Fq.ec`

### 7.3.2 Loop Structure Analysis

The three-level nested loop has the following structure:

**Outer loop** `while (len > 0)`: iterates  $\ell \in \{128, 64, 32, 16, 8, 4, 2, 1\}$ , corresponding to 8 NTT stages.

**Middle loop** `while (start < 256)`: for each stage, iterates over the  $256/(2\ell)$  butterfly groups. At stage  $s$ , there are  $2^s$  groups.

**Inner loop** `while (j < start + len)`: performs  $\ell$  butterfly operations within each group, all using the same twiddle factor `zeta = zetas[k]`.

The twiddle counter  $k$  advances once per group (per iteration of the middle loop), consuming a total of  $\sum_{s=0}^7 2^s = 255$  twiddle factors.

### 7.3.3 Loop Invariants

**Definition 7.2** (Outer loop invariant  $I_{\text{out}}$ ). After completing all stages with  $\ell' > \ell$ :

1.  $k = 256/\ell$ : the twiddle counter has consumed all factors for completed stages.
2. For all  $i \in [0, 256)$ , the coefficient  $a[i]$  is partially transformed: it equals the sum of the appropriate butterfly outputs through stage  $8 - \log_2(\ell)$ .
3. For all  $i \in [0, 256)$ ,  $|a[i]| < 2^{15+(8-\log_2 \ell)}$ .

**Definition 7.3** (Middle loop invariant  $I_{\text{mid}}$ ). After completing all groups with  $start' < start$  in stage  $s$ :

1. Indices in  $[0, start)$  have been processed by stage  $s$ .
2.  $k = 256/\ell + start/(2\ell)$ : twiddle counter accounts for completed groups.
3. Bounds are maintained as in  $I_{\text{out}}$ .

**Definition 7.4** (Inner loop invariant  $I_{\text{in}}$ ). After completing  $j - start$  butterflies in group  $[start, start + 2\ell)$ :

1. Indices in  $[start, j)$  and  $[start + \ell, j + \ell)$  have been updated by the butterfly with factor  $\zeta = \mathbf{zet}\mathbf{as}[k - 1]$ .
2. Indices in  $[j, start + \ell)$  are unchanged from before this stage.
3. Bounds:  $|a[i]| < 2^{16+s}$  for updated indices.

### 7.3.4 Termination

**Lemma 7.5** (NTT termination). *The procedure `NTT.ntt` terminates for any input array.*

*Proof.* Each outer loop iteration halves `len`, so after at most 8 iterations, `len` = 0 and the loop exits. For each fixed `len`, the middle loop increments `start` by  $2 \cdot \mathbf{len}$ , so it terminates after at most  $256/(2 \cdot \mathbf{len})$  iterations. For each fixed `start`, the inner loop increments `j` by 1, terminating after exactly `len` iterations.

The variant functions are:

- Outer: `len` (decreases each iteration).
- Middle:  $256 - \mathbf{start}$  (decreases by  $2 \cdot \mathbf{len}$  each iteration).
- Inner:  $\mathbf{start} + \mathbf{len} - j$  (decreases by 1 each iteration).

□

## 7.4 Inverse NTT Procedure

### 7.4.1 Specification

The inverse NTT is specified as the EasyCrypt procedure `NTT.invntt`:

```

1 module NTT = {
2   (* ... ntt as above ... *)
3
4   proc invntt(a : int Array256.t) : int Array256.t = {
5     var len, start, j, k : int;
```

```

6   var t, zeta          : int;
7   var f                : int;
8
9   f <- -29720;  (* Montgomery form of 1/256 *)
10  k <- 255;
11  len <- 1;
12  while (len < 256) {
13    start <- 0;
14    while (start < 256) {
15      zeta <- zetas_inv.[k];
16      k <- k - 1;
17      j <- start;
18      while (j < start + len) {
19        t <- a.[j];
20        a.[j] <- t + a.[j + len];
21        a.[j + len] <- montgomery_reduce (zeta * (t - a
22          .[j + len]));
23        j <- j + 1;
24      }
25      start <- start + 2 * len;
26    }
27    len <- len * 2;
28  }
29  j <- 0;
30  while (j < 256) {
31    a.[j] <- montgomery_reduce (f * a.[j]);
32    j <- j + 1;
33  }
34  return a;
35 }.

```

Listing 7.2: Inverse NTT specification in NTT\_Fq.ec

### 7.4.2 Differences from the Forward NTT

The inverse NTT differs from the forward NTT in the following ways:

1. The outer loop runs  $\ell$  from 1 to 128 (increasing, rather than decreasing from

128 to 1).

2. The twiddle counter  $k$  *decreases* from 255 to 1.
3. The butterfly is “gentle-side-up”: the update to  $a[j + \ell]$  includes a Montgomery reduction, while the update to  $a[j]$  is a plain addition.
4. An extra final loop scales every coefficient by  $f = -29720$  via Montgomery reduction.

### 7.4.3 Final Scaling

**Lemma 7.6** (Final scaling correctness). *Let  $a \in \mathbb{Z}^{256}$  be the array after the main inverse NTT loops, with  $|a[i]| < 2^{23}$ . After the final scaling loop:*

$$a'[i] \equiv a[i] \cdot (256)^{-1} \cdot R \pmod{q}$$

and  $|a'[i]| < q/2$  for all  $i$ .

*Proof.* The constant  $f = -29720$  satisfies  $f \equiv -\text{MONT}^2 \cdot 256^{-1} \pmod{q}$ . Thus:

$$\begin{aligned} \text{mont\_reduce}(f \cdot a[i]) &\equiv f \cdot a[i] \cdot R^{-1} \pmod{q} \\ &\equiv (-R^2 \cdot 256^{-1}) \cdot a[i] \cdot R^{-1} \pmod{q} \\ &\equiv -R \cdot 256^{-1} \cdot a[i] \pmod{q}. \end{aligned}$$

The negative sign is absorbed by the fact that the inverse NTT formula itself has an implicit sign from the negacyclic structure. The bound  $|a'[i]| < q/2$  follows from Theorem 6.3.  $\square$

## 7.5 Round-Trip Theorem at the Imperative Level

**Theorem 7.7** (Imperative round-trip). *For any input array  $a$  with  $|a[i]| < 2^{15}$  for all  $i$ :*

$$\text{NTT.invntt}(\text{NTT.ntt}(a)) \equiv a \pmod{q} \text{ (pointwise).}$$

*Proof sketch.* By composition of correctness lemmas:

1.  $\text{NTT.ntt}(a) = \hat{a}$  where  $\hat{a}[i] = \text{NTT}(a)[i]$  (from the forward NTT correctness, proved in Chapter 8).
2.  $\text{NTT.invntt}(\hat{a}) = a'$  where  $a'[i] = \text{INTT}(\hat{a})[i] \cdot R \bmod q$  (from the inverse NTT correctness with the Montgomery factor).
3.  $\text{INTT}(\text{NTT}(a)) = a$  (abstract round-trip, from Proposition 2.1).
4. The Montgomery factor  $R$  is absorbed by the representation convention in the inverse NTT output.

The full proof is completed in `NTTEndToEnd.ec`. □

## 7.6 Twiddle Factor Synchronisation

A subtle but critical aspect of the imperative specification is the *twiddle factor synchronisation*: the counter  $k$  must be in exact correspondence with the current loop indices  $(\ell, \text{start})$  at every point in the algorithm. This invariant is captured formally as:

**Lemma 7.8** (Twiddle counter synchronisation). *At any point during the execution of `NTT.ntt`, with current values  $\ell_{\text{cur}}$  and  $\text{start}_{\text{cur}}$ :*

$$k = \frac{256}{\ell_{\text{cur}}} + \frac{\text{start}_{\text{cur}}}{2\ell_{\text{cur}}}.$$

This lemma is essential for connecting the imperative specification to the abstract spectral specification, where twiddle factors are indexed directly by the group number.



# Chapter 8

## Algebraic NTT Infrastructure

### 8.1 Overview of `NTTFullSpec.ec` and `NTTFullAlgebra.ec`

The abstract spectral NTT specification lives in two files:

- `NTTFullSpec.ec` (160 lines) defines the abstract evaluation maps `full_ntt` and `full_invntt` and states the main correctness lemmas.
- `NTTFullAlgebra.ec` (1000+ lines) develops the algebraic machinery needed to prove those lemmas: primitive root properties, bit-reversal identities, and butterfly-stage decompositions.

### 8.2 Primitive Root Properties

#### 8.2.1 Existence and Uniqueness

The prime  $q = 64513$  satisfies  $q \equiv 1 \pmod{512}$ . Therefore  $\mathbb{F}_q^*$  contains an element of order 512, i.e., a primitive 512th root of unity.

**Theorem 8.1** (Primitive root existence). *The element  $\zeta_0 = 426 \in \mathbb{F}_q$  has multiplicative order exactly 512.*

*Proof.* We verify:

1.  $\zeta_0^{512} \equiv 1 \pmod{64513}$  (direct computation).

2. For all  $d \mid 512$  with  $d < 512$ :  $\zeta_0^d \not\equiv 1 \pmod{64513}$ . It suffices to check  $d = 256$  and  $d = 128$  (since  $512 = 2^9$  and the order must divide 512).

Both computations are discharged by the `smt` tactic. □

**Corollary 8.2.**  $\zeta_0^{256} \equiv -1 \pmod{q}$  and  $\zeta_0^{128} \not\equiv \pm 1 \pmod{q}$ .

## 8.2.2 Orthogonality

The 512th roots of unity in  $\mathbb{F}_q$  are  $\{\zeta_0^k : k = 0, 1, \dots, 511\}$ . The primitive roots are those  $\zeta_0^k$  with  $\gcd(k, 512) = 1$ . The NTT formula uses the set of *odd* powers  $\{\zeta_0^{2i+1} : i = 0, \dots, 255\}$ , which are exactly the primitive roots.

**Lemma 8.3** (Orthogonality relation). *For  $i, j \in \{0, \dots, 255\}$  with  $i \neq j$ :*

$$\sum_{k=0}^{255} \zeta_0^{(2i+1)k} \cdot \zeta_0^{-(2j+1)k} = 0 \in \mathbb{F}_q.$$

*For  $i = j$ , the sum equals 256.*

*Proof.* For  $i \neq j$ : the sum is a geometric series with ratio  $r = \zeta_0^{2(i-j)}$ . Since  $\zeta_0$  has order 512,  $r$  is a 256th root of unity. The exponent  $2(i-j) \not\equiv 0 \pmod{256}$  (because  $|i-j| < 256$ , so  $2|i-j| < 512$ , and  $2(i-j) \neq 0$ ). Thus  $r^{256} = \zeta_0^{512(i-j)} = 1$  and  $r \neq 1$ , so  $\sum_{k=0}^{255} r^k = (r^{256} - 1)/(r - 1) = 0$ .

For  $i = j$ : each term is  $\zeta_0^0 = 1$ , and there are 256 terms. □

## 8.3 Bit-Reversal Permutation

### 8.3.1 Definition

The bit-reversal permutation  $\text{brev}_8 : \{0, \dots, 255\} \rightarrow \{0, \dots, 255\}$  reverses the 8 binary digits of its input:

$$\text{brev}_8(b_7b_6b_5b_4b_3b_2b_1b_0) = b_0b_1b_2b_3b_4b_5b_6b_7.$$

```
1 op bsrev (n k : int) : int =
2   bsrev8 k.      (* 8-bit bit reversal *)
```

```

3
4 op bsrev8 (k : int) : int =
5   let k0 = k %% 2 in
6   let k1 = (k %/ 2) %% 2 in
7   (* ... *)
8   let k7 = (k %/ 128) %% 2 in
9   k0 * 128 + k1 * 64 + k2 * 32 + k3 * 16 +
10  k4 * 8 + k5 * 4 + k6 * 2 + k7.

```

Listing 8.1: Bit-reversal definition in NTTFullAlgebra.ec

### 8.3.2 Key Properties

**Lemma 8.4** (Bit-reversal involutivity).  $\text{brev}_8(\text{brev}_8(k)) = k$  for all  $k \in [0, 256)$ .

**Lemma 8.5** (Bit-reversal and halving). For  $k \in [1, 256)$  and  $\ell = 2^s$  (a power of 2 with  $s \leq 7$ ):

$$\text{brev}_8(k \bmod \ell) = \text{brev}_8(k) / (256/\ell) \cdot (256/\ell) + \text{brev}_8(k) \bmod (256/\ell).$$

This lemma, though technical, is essential for connecting the loop index  $k$  to the spectral index  $\text{brev}_8(k)$ .

### 8.3.3 Bit-Reversal and Twiddle Factor Indexing

**Lemma 8.6** (Twiddle factor ordering). In the forward NTT outer loop with half-length  $\ell = 2^{7-s}$ , after completing  $g$  groups of stage  $s$ , the twiddle counter satisfies  $k = 2^s + g$ , and the twiddle factor is  $\zeta_0^{\text{brev}_8(2^s+g)}$ .

*Proof.* By induction on  $s$  (outer loops) and  $g$  (groups within a stage). The key observation is that as the outer counter increments, the bit-reversal of the counter gives exactly the correct spectral index for the butterfly group.  $\square$

## 8.4 Stage Decomposition Theorem

### 8.4.1 Partial NTT

**Definition 8.7** (Stage- $s$  partial NTT). For  $0 \leq s \leq 8$ , the stage- $s$  partial NTT of a vector  $f \in \mathbb{F}_q^{256}$  is defined by:

$$\text{NTT}_s(f)[i] = \sum_{j \equiv i \pmod{2^s}} f[j] \cdot \zeta_0^{\text{brev}_s(\lfloor i/2^s \rfloor) \cdot j}, \quad i = 0, \dots, 255.$$

The full NTT corresponds to  $\text{NTT}_8$  (all 8 stages completed). The identity  $\text{NTT}_0(f) = f$  holds trivially (the empty product is 1).

### 8.4.2 Butterfly Stage Lemma

**Lemma 8.8** (Single-stage butterfly correctness). For all  $s \in \{0, \dots, 7\}$  and  $f \in \mathbb{F}_q^{256}$ :

$$\text{NTT}_{s+1}(f) = \text{butterfly\_stage}(s, \text{NTT}_s(f)),$$

where  $\text{butterfly\_stage}(s, a)$  applies one complete pass of butterflies with half-length  $\ell = 2^{7-s}$ .

*Proof.* Fix an index  $i$  and let  $g = \lfloor i/2^{s+1} \rfloor$  (the group number),  $r = i \bmod 2^{s+1}$  (the position within the group),  $r_0 = r \bmod 2^s$  (the low-half position).

The butterfly for group  $g$  uses twiddle factor  $w = \zeta_0^{\text{brev}_s(2^s + g)}$ .

Case  $r < 2^s$  (upper butterfly input):

$$\begin{aligned} \text{butterfly\_stage}(s, a)[i] &= a[i] + w \cdot a[i + 2^s] \\ &= \sum_{j \equiv r_0 \pmod{2^s}} f[j] \zeta_0^{\text{brev}_s(g)j} + \zeta_0^{\text{brev}_s(2^s + g)} \sum_{j \equiv r_0 \pmod{2^s}} f[j + 2^s] \zeta_0^{\text{brev}_s(g)j}. \end{aligned}$$

(By the inductive hypothesis  $\text{NTT}_s(f)[i] = \sum_{j \equiv i \pmod{2^s}} f[j] \zeta_0^{\text{brev}_s(\lfloor i/2^s \rfloor)j}$ .)

Combining the sums over  $j$  and  $j + 2^s$ , and using the identity  $\text{brev}_s(2^s + g) = \text{brev}_s(g)/2 + 2^{7-s} \cdot (g \bmod 2^s)$  (from Lemma 8.5), one can verify that the result equals  $\text{NTT}_{s+1}(f)[i]$  by direct algebraic manipulation.

The case  $r \geq 2^s$  is symmetric. The full algebraic proof in EasyCrypt uses the `ring` tactic after reducing to finite sums.  $\square$

### 8.4.3 Full NTT Correctness

**Theorem 8.9** (Forward NTT correctness). *The imperative procedure `NTT.ntt` computes the full NTT:*

$$\forall a \in \mathbb{Z}_q^{256}. \text{NTT.ntt}(a) = \text{NTT}_8(a) = \text{full\_ntt}(a).$$

*Proof.* By induction on the outer loop iteration, using Lemma 8.8 as the inductive step. The loop invariant asserts that after  $s$  outer loop iterations, the array equals  $\text{NTT}_s$  of the input.

The base case ( $s = 0$ ): the array is unchanged, matching  $\text{NTT}_0 = \text{id}$ . The inductive step: applying the butterfly stage transforms  $\text{NTT}_s$  to  $\text{NTT}_{s+1}$  by Lemma 8.8. The final case ( $s = 8$ ):  $\text{NTT}_8 = \text{full\_ntt}$  by Definition 4.1.  $\square$

## 8.5 Inverse NTT Algebra

### 8.5.1 Abstract Inverse NTT

**Definition 8.10** (Abstract inverse NTT, `full_invntt`).

$$\text{INTT}(p)[i] = 256^{-1} \sum_{j=0}^{255} p[j] \cdot \zeta_0^{-((2\text{brev}_8(j)+1) \cdot i)}$$

**Theorem 8.11** (Inverse NTT correctness).  $\text{full\_invntt}(\text{full\_ntt}(p)) = p$  for all  $p \in \mathbb{Z}_q^{256}$ .

*Proof.* Substituting the definition of `full_ntt`:

$$\begin{aligned}
 \text{INTT}(\text{NTT}(p))[i] &= 256^{-1} \sum_{j=0}^{255} \text{NTT}(p)[j] \cdot \zeta_0^{-((2\text{brev}_8(j)+1)i)} \\
 &= 256^{-1} \sum_{j=0}^{255} \left( \sum_{k=0}^{255} p[k] \cdot \zeta_0^{(2\text{brev}_8(j)+1)k} \right) \cdot \zeta_0^{-((2\text{brev}_8(j)+1)i)} \\
 &= 256^{-1} \sum_{k=0}^{255} p[k] \sum_{j=0}^{255} \zeta_0^{(2\text{brev}_8(j)+1)(k-i)}.
 \end{aligned}$$

For  $k = i$ : the inner sum is  $\sum_{j=0}^{255} 1 = 256$ . For  $k \neq i$ : the inner sum is 0 by a variant of Lemma 8.3 (substituting  $m = \text{brev}_8(j)$  and using that  $\text{brev}_8$  is a bijection on  $[0, 256)$ ).

Thus  $\text{INTT}(\text{NTT}(p))[i] = 256^{-1} \cdot p[i] \cdot 256 = p[i]$ .  $\square$

## 8.6 The Critical Difference from MLKEM

**Remark 8.12** (HAETAE vs. MLKEM exponent structure). In MLKEM, the twiddle factor for position  $(s, g)$  in the loop is  $\zeta^{\text{brev}_8(2g+1)}$  (with a  $2g + 1$  structure), which corresponds to the *incomplete* 128-point transform used in that scheme. The algebra lemmas for MLKEM are stated in terms of  $\text{brev}_7$  (7-bit reversal) rather than  $\text{brev}_8$ .

In HAETAE, the twiddle factor is  $\zeta^{\text{brev}_8(2^s+g)}$ , where the bit-reversal acts on an index that includes the stage counter. This produces the *complete* 256-point negacyclic NTT. The corresponding algebra lemmas require  $\text{brev}_8$  (8-bit reversal) and the orthogonality relation over all 256 primitive 512th roots of unity.

This distinction is the primary obstacle to reusing MLKEM’s EasyCrypt proofs, and is the motivation for developing `NTTFullAlgebra.ec` from scratch.

# Chapter 9

## Jasmin Implementation

### 9.1 Overview

The high-assurance implementation of the HAETAE NTT is written in the **Jasmin** programming language. Three source files collectively implement the forward NTT, inverse NTT, and base multiplication:

- `reduce.jinc`: Montgomery reduction and field multiplication.
- `zetas.jinc`: Precomputed twiddle factor table.
- `poly.jinc`: Forward and inverse NTT kernels.
- `hpoly.jazz`: Exported entry points callable from C.

The **Jasmin** source is compiled to x86-64 assembly (`hpoly.s`) and simultaneously extracted to an **EasyCrypt** module for formal verification.

### 9.2 Montgomery Reduction in Jasmin

#### 9.2.1 Source Code

```
1 param int JPRIME = 64513;  
2 param int JQINV  = 940508161;  
3
```

```

4 inline fn __montgomery_reduce(reg u64 a) -> reg u32 {
5     reg u64 t64;
6     reg u32 t, r;
7
8     t      = (32u)(a);           /* low 32 bits of a */
9     t      *= (u32)(JQINV);      /* t = (a mod 2^32) * QINV mod
10                                2^32 */
11     t64    = (64s)(t);          /* sign-extend to 64 bits */
12     t64    *= (u64)(JPRIME);     /* t64 = t * q (64-bit) */
13     t64    = a - t64;           /* t64 = a - t*q */
14     r      = (32u)(t64 >> 32);  /* r = (a - t*q) / 2^32 (
15                                arithmetic shift) */
16     return r;
17 }

```

Listing 9.1: Montgomery reduction in `reduce.jinc`

## 9.2.2 Jasmin Semantics

The Jasmin code uses explicit types to control the width and signedness of arithmetic:

- `(32u)(a)`: truncate 64-bit value to 32 bits (unsigned).
- `(u32)(JQINV)`: interpret the literal as a 32-bit unsigned value.
- `(64s)(t)`: sign-extend a 32-bit value to 64 bits.
- `(32u)(t64 >> 32)`: arithmetic right shift and truncate.

The `>>` operator on a signed 64-bit value in Jasmin performs an *arithmetic* right shift, matching the C behaviour for signed integers.

## 9.2.3 Correspondence to C

The Jasmin code directly implements the C reference in ?? 6.1:

- `(32u)(a) * (u32)(JQINV)` corresponds to `(int32_t)a * (int32_t)QINV`.
- The sign extension `(64s)(t)` followed by multiplication by `JPRIME` corresponds to `(int64_t)t * Q`.

- The arithmetic right shift  $\gg 32$  corresponds to  $\gg 32$  on a signed value.

## 9.3 Field Multiplication

```

1 inline fn __fqmul(reg u32 a, reg u32 b) -> reg u32 {
2     reg u64 t64;
3     reg u32 r;
4
5     t64 = (64s)(a);
6     t64 *= (64s)(b);
7     r    = __montgomery_reduce(t64);
8     return r;
9 }

```

Listing 9.2: Field multiplication in `reduce.jinc`

This computes  $a \cdot b \cdot R^{-1} \bmod q$  where both  $a$  and  $b$  are expected to be in the range  $(-2^{15}, 2^{15})$ , so their 64-bit product does not overflow.

## 9.4 Forward NTT Kernel

### 9.4.1 Source Code

```

1 inline fn _poly_ntt(reg mut ptr u32[256] rp) -> reg ptr
2     u32[256] {
3     reg u32 zeta, t;
4     reg u64 t64;
5     reg int len, start, j, k;
6
7     k = 1;
8     len = 128;
9
10    while (len > 0) {
11        start = 0;
12        while (start < 256) {
13            zeta = (u32) jzetas[k];
14            k += 1;

```

```

14
15     j = start;
16     while (j < start + len) {
17         t64 = (64s)(zeta);
18         t64 *= (64s)(rp[j + len]);
19         t    = __montgomery_reduce(t64);
20
21         rp[j + len] = rp[j] - t;
22         rp[j]       = rp[j] + t;
23         j += 1;
24     }
25     start += 2 * len;
26 }
27 len >>= 1;
28 }
29 return rp;
30 }

```

Listing 9.3: Forward NTT kernel in `poly.jinc`

## 9.4.2 Structural Correspondence

The Jasmin kernel `_poly_ntt` has a direct structural correspondence to the C reference (?? 3.1) and the EasyCrypt specification (?? 7.1):

| Aspect      | C reference                                                  | Jasmin                                                 |
|-------------|--------------------------------------------------------------|--------------------------------------------------------|
| Outer loop  | <code>for (len = 128; len &gt; 0; len &gt;&gt;= 1)</code>    | <code>while (len &gt; 0) {...; len &gt;&gt;= 1}</code> |
| Middle loop | <code>for (start = 0; start &lt; 256; start += 2*len)</code> | equivalent <code>while</code>                          |
| Inner loop  | <code>for (j = start; j &lt; start+len; j++)</code>          | equivalent <code>while</code>                          |
| Twiddle     | <code>zetas[k++]</code>                                      | <code>jzetas[k]; k += 1</code>                         |
| Butterfly   | <code>t = mont_reduce(zeta * a[j+len])</code>                | same                                                   |

The key difference is the use of explicit type casts in Jasmin to manage word widths.

### 9.4.3 Array Access Pattern

In Jasmin, the global constant table `jzetas` is accessed as a `u32` array. The entry at index  $k$  is the Montgomery form of the  $k$ -th twiddle factor. The access pattern `jzetas[k]` with  $k$  starting at 1 and incrementing each iteration of the middle loop ensures that consecutive twiddle factors are consumed in order.

## 9.5 Inverse NTT Kernel

```

1  inline fn _poly_invntt(reg mut ptr u32[256] rp) -> reg ptr
    u32[256] {
2      reg u32 zeta, t, f;
3      reg u64 t64;
4      reg int len, start, j, k;
5
6      k = 255;
7      len = 1;
8
9      while (len < 256) {
10         start = 0;
11         while (start < 256) {
12             zeta = (u32) jzetas_inv[k];
13             k -= 1;
14
15             j = start;
16             while (j < start + len) {
17                 t = rp[j];
18                 rp[j] = t + rp[j + len];
19
20                 t64 = (64s)(zeta);
21                 t64 *= (64s)(t - rp[j + len]);
22                 rp[j + len] = __montgomery_reduce(t64);
23
24                 j += 1;
25             }
26             start += 2 * len;
27         }
28         len <<= 1;

```

```

29     }
30
31     /* Final scaling */
32     f = (u32) jzetas_inv[255];
33     j = 0;
34     while (j < 256) {
35         t64 = (64s)(f);
36         t64 *= (64s)(rp[j]);
37         rp[j] = __montgomery_reduce(t64);
38         j += 1;
39     }
40     return rp;
41 }

```

Listing 9.4: Inverse NTT kernel in `poly.jinc` (condensed)

### 9.5.1 Inverse Butterfly Structure

The inverse butterfly in Jasmin computes:

$$\begin{aligned}
 t &\leftarrow \text{rp}[j] \\
 \text{rp}[j] &\leftarrow t + \text{rp}[j + \ell] \\
 \text{rp}[j + \ell] &\leftarrow \text{mont}(\zeta \cdot (t - \text{rp}[j + \ell]))
 \end{aligned}$$

This is the inverse (Gentleman–Sande) butterfly. It is the algebraic inverse of the Cooley–Tukey butterfly: if the forward butterfly computes  $(a + b, a - b)$  with twiddle  $w$  applied to the lower output, the inverse butterfly computes  $(a + b, w(a - b))$  and is composed with the twiddle application on the upper output in the forward case.

## 9.6 Exported Entry Points

```

1 export fn poly_ntt_jazz(reg mut ptr u32[256] rp)
2     -> reg ptr u32[256] {
3     rp = _poly_ntt(rp);
4     return rp;
5 }
6

```

```

7 export fn poly_invntt_jazz(reg mut ptr u32[256] rp)
8   -> reg ptr u32[256] {
9     rp = _poly_invntt(rp);
10    return rp;
11  }
12
13 export fn poly_basemul_jazz(
14   reg mut ptr u32[256] rp,
15   reg ptr u32[256] ap,
16   reg ptr u32[256] bp)
17   -> reg ptr u32[256] {
18     rp = _poly_basemul(rp, ap, bp);
19     return rp;
20  }

```

Listing 9.5: Exported functions in `hpoly.jazz`

The `export` keyword triggers both assembly generation and `EasyCrypt` extraction. The ABI of `poly_ntt_jazz` takes a pointer to a 256-word array in a register and returns the same pointer after in-place modification.

## 9.7 Safety Verification

The Jasmin safety checker verifies that all array accesses are in bounds:

- `rp[j]` and `rp[j+len]` are accessed with  $j \in [\text{start}, \text{start} + \text{len})$  and  $j + \text{len} \in [\text{start} + \text{len}, \text{start} + 2\text{len})$ . Both ranges are within  $[0, 256)$  by the loop bounds.
- `jzetas[k]` is accessed with  $k \in [1, 256)$ , which is within the bounds of the 256-element twiddle table.

All safety conditions are discharged automatically by the `Jasmin` compiler’s safety analysis.

## 9.8 Generated Assembly

The compiled assembly `hpoly.s` contains straightforward x86-64 code. Key assembly features include:

- Register allocation: loop variables (`len`, `start`, `j`, `k`) are assigned to general-purpose registers `r8–r15`.
- Montgomery reduction uses `imul` (signed 64-bit multiply) and `sar` (arithmetic shift right) instructions.
- No SIMD instructions are used in the scalar implementation; a vectorised (AVX2) version is possible but beyond the scope of this dissertation.

The correspondence between the **Jasmin** source, the generated assembly, and the **EasyCrypt** extraction is guaranteed by the **Jasmin** compiler correctness theorem [Alm+17], which is itself a machine-checked proof in the Coq proof assistant.

# Chapter 10

## Jasmin-to-EasyCrypt Refinement Proof

### 10.1 Overview of RefJasminNTT.ec

The file `RefJasminNTT.ec` (1000+ lines) bridges the gap between the machine-level Jasmin implementation and the abstract algorithmic specification. It is the most technically demanding proof file in the development, requiring simultaneous reasoning about:

1. Signed vs. unsigned word arithmetic.
2. Array representation correspondences.
3. Integer arithmetic bounds.
4. Equivalence of three-level nested loops.

### 10.2 Word-to-Coefficient Correspondence

#### 10.2.1 The Linking Predicate

The central predicate connecting machine words to mathematical integers is:

```
1 pred wrel (rp : W32.t Array256.t) (a : int Array256.t) =  
2   forall i, 0 <= i < 256 =>
```

```
3  W32.to_sint rp.[i] = a.[i].
```

Listing 10.1: Array correspondence predicate in RefJasminNTT.ec

This predicate asserts that the 32-bit word representation of each element in `rp` equals the integer in `a`. Note that `W32.to_sint` applies the two's-complement signed interpretation.

## 10.2.2 Preservation Under Arithmetic

**Lemma 10.1** (Word addition preservation). *If  $wrel(rp, a)$  and  $|a[i]|, |b[i]| < 2^{30}$ , then  $wrel(rp \oplus_{32} b, a + b)$  where  $\oplus_{32}$  denotes 32-bit word addition.*

*Proof.* The key identity is: for  $a, b \in \mathbb{Z}$  with  $|a|, |b| < 2^{30}$ ,

$$W32.to\_sint(W32.of\_int(a) + W32.of\_int(b)) = a + b,$$

which holds because the 32-bit arithmetic does not overflow when both operands are bounded by  $2^{30}$ .  $\square$

## 10.3 Montgomery Reduction Correspondence

### 10.3.1 Word-Level Reduction

The `Jasmin` procedure computes Montgomery reduction on machine words. The correspondence lemma asserts that it equals the abstract integer-level computation:

**Lemma 10.2** (Montgomery reduction word correspondence). *Let  $z \in \mathbb{Z}$  with  $|z| < 2^{31} \cdot q$ . Then:*

$$W32.to\_sint(Extracted.montgomery\_reduce(W64.of\_int(z))) = NTT\_Fq.montgomery\_reduce(z).$$

*Proof.* The extracted procedure performs the same arithmetic as the integer-level specification. The key step is showing that intermediate values do not overflow: the 64-bit product  $m \cdot q$  satisfies  $|mq| \leq (2^{32} - 1) \cdot 64513 < 2^{63}$ , fitting in a signed 64-bit register.  $\square$

## 10.4 Butterfly Equivalence

### 10.4.1 Individual Butterfly

**Lemma 10.3** (Single butterfly correspondence). *Let  $\mathbf{rp}, \mathbf{a}$  satisfy  $wrel(\mathbf{rp}, \mathbf{a})$ , and let  $|a[j]|, |a[j + \ell]| < 2^{15}$  and  $|\zeta| < 2^{15}$ . Then after executing one butterfly step:*

$$wrel(\mathbf{rp}', \mathbf{a}'),$$

where  $\mathbf{a}'$  is the result of the abstract butterfly and  $\mathbf{rp}'$  is the result of the machine butterfly.

*Proof.* The machine butterfly computes:

$$\begin{aligned} t &= W32(\text{Extracted.mont\_reduce}(\zeta \cdot \mathbf{rp}[j + \ell])) \\ \mathbf{rp}'[j + \ell] &= \mathbf{rp}[j] - t \\ \mathbf{rp}'[j] &= \mathbf{rp}[j] + t \end{aligned}$$

By Lemma 10.2,  $W32.to\_sint(t) = \text{mont}(\zeta \cdot a[j + \ell])$ . By Lemma 10.1, the addition/-subtraction preserves the correspondence (using bounds  $|a[j]|, |\text{mont}(\zeta \cdot a[j + \ell])| < 2^{16}$ , so their sum/difference is bounded by  $2^{17} < 2^{30}$ , safe in 32-bit words).  $\square$

## 10.5 Loop Equivalence Proofs

### 10.5.1 Inner Loop Equivalence

**Lemma 10.4** (Inner loop correspondence). *Given  $wrel(\mathbf{rp}, \mathbf{a})$  and bounds  $|a[i]| < 2^{15}$ :*

$$hoare[\text{Extracted.inner\_loop} \sim NTT\_Fq.\text{inner\_loop} : wrel(\mathbf{rp}, \mathbf{a}) \Rightarrow wrel(\mathbf{rp}', \mathbf{a}')].$$

*Proof.* By induction on the inner loop counter  $j$ , using Lemma 10.3 as the inductive step. The bound on coefficients increases by at most  $2^{16}$  per butterfly (from Theorem 6.3), but the inner loop uses the same twiddle factor throughout, so the number of butterflies per inner loop execution is exactly  $\ell$ .  $\square$

## 10.5.2 Full NTT Loop Equivalence

**Theorem 10.5** (Forward NTT loop equivalence). *Let  $rp$  satisfy  $|a[i]| < 2^{15}$  for all  $i$ . Then:*

$$\begin{aligned} & \text{hoare}[\text{Extracted.poly\_ntt\_jazz} \sim \text{NTT\_Fq.NTT.ntt} : \\ & \quad \text{wrel}(rp, a) \\ & \quad \Rightarrow \text{wrel}(rp', a') \wedge \forall i. |a'[i]| < 2^{23}]. \end{aligned}$$

*Proof.* By three-level loop induction:

1. The outer loop induction uses  $\ell$  as the decreasing variant. The invariant after completing all stages with  $\ell' > \ell$  is the partial NTT condition.
2. The middle loop induction uses  $256 - \text{start}$  as the decreasing variant.
3. The inner loop uses Lemma 10.4.

The bound  $|a'[i]| < 2^{23}$  follows from Theorem 6.6. □

## 10.6 Main Refinement Lemmas

The two main lemmas of RefJasminNTT.ec are:

```

1  (* Forward NTT: Jasmin core ~ NTT_Fq, bounds 16 -> 24 *)
2  lemma poly_ntt_core_ref rp a :
3    wrel rp a =>
4    (forall i, 0 <= i < 256 => '|a.[i]| < 2^15) =>
5    equiv[Extracted.poly_ntt_jazz ~ NTT_Fq.NTT.ntt :
6      ={arg} /\ wrel arg{1} arg{2} ==>
7      wrel res{1} res{2} /\
8      forall i, 0 <= i < 256 => '|res{2}.[i]| < 2^23}.
9
10 (* Inverse NTT: Jasmin core ~ NTT_Fq, bounds 16 -> 16,
11    Montgomery *)
12 lemma poly_invntt_core_ref rp a :
13   wrel rp a =>

```

```

13  (forall i, 0 <= i < 256 => '|a.[i]| < 2^23) =>
14  equiv[Extracted.poly_invntt_jazz ~ NTT_Fq.NTT.invntt :
15    ={arg} /\ wrel arg{1} arg{2} ==>
16    wrel res{1} res{2} /\
17    forall i, 0 <= i < 256 => '|res{2}.[i]| < 2^15].

```

Listing 10.2: Main refinement lemmas in RefJasminNTT.ec

These two lemmas represent the central contribution of RefJasminNTT.ec. Once proved, they serve as the foundation for the end-to-end theorem.

## 10.7 Proof Engineering Challenges

### 10.7.1 Signed vs. Unsigned Arithmetic

The Jasmin code uses `u32` (unsigned) for coefficient storage but `(64s)(x)` (signed 64-bit) for intermediate products. The EasyCrypt extraction mirrors this by using `W32.t` for storage and `W64.t` for products. The refinement proof must track the signed interpretation of unsigned words, which requires the signed-arithmetic lemmas from Fq.ec.

### 10.7.2 64-bit Product Non-Overflow

The 64-bit product  $\zeta \cdot a[j + \ell]$  requires:  $|\zeta| \cdot |a[j + \ell]| < 2^{63}$ . Since  $|\zeta| < q < 2^{16}$  and  $|a[j + \ell]| < 2^{23}$  (after the forward NTT), we have  $|\zeta \cdot a[j + \ell]| < 2^{39} \ll 2^{63}$ . This is verified in the proof as a side condition.

### 10.7.3 Twiddle Factor Table Agreement

The EasyCrypt extracted module accesses the twiddle table as a global constant. The refinement proof must show that this global constant agrees with the `zetas` array defined in NTT\_Fq.ec. This is established by a computation lemma:

**Lemma 10.6** (Twiddle table agreement). *For all  $k \in [0, 256)$ :  $W32.to\_sint(jzetas[k]) = NTT\_Fq.zetas[k]$ .*

*Proof.* Both tables are defined by the same formula modulo  $q = 64513$ . Since  $q < 2^{16}$ , all entries fit in 16 bits and the signed interpretation agrees with the unsigned value. Verified by the SMT backend.  $\square$

#### 10.7.4 In-Place Update Semantics

Both the **Jasmin** implementation and the **EasyCrypt** specification modify the input array in place. The refinement proof tracks the state of each array cell individually, using the **EasyCrypt** array theory lemmas to reason about functional updates.

The key identity is: after an in-place butterfly that modifies indices  $j$  and  $j + \ell$ , all other indices remain unchanged. This is expressed as:

$$\forall i \neq j, i \neq j + \ell. \text{rp}'[i] = \text{rp}[i] \wedge \text{a}'[i] = \text{a}[i].$$

# Chapter 11

## End-to-End Correctness

### 11.1 Overview of NTTEndToEnd.ec

The file `NTTEndToEnd.ec` (105 lines) is the apex of the verification chain. It imports all preceding modules and states the final correctness theorems that directly relate the exported Jasmin functions to the abstract spectral NTT specification.

```
1 require import NTTFullSpec NTTFullAlgebra .
2 require import NTT_Fq Fq GFq Rq .
3 require import RefJasminNTT .
4 require import Hpoly_extract .
```

Listing 11.1: Module imports in `NTTEndToEnd.ec`

### 11.2 Main Correctness Theorems

#### 11.2.1 Forward NTT Correctness

**Theorem 11.1** (Jasmin NTT correctness). *Let  $\mathbf{rp} : \mathbf{W32}.t^{256}$  be a word array encoding a polynomial with 16-bit signed coefficients (i.e.,  $|\mathbf{W32.to\_sint}(\mathbf{rp}[i])| < 2^{15}$  for all  $i$ ). Then executing `poly_ntt_jazz(rp)` produces an output array  $\mathbf{rp}'$  satisfying:*

$$\mathbf{W32.to\_sint}(\mathbf{rp}'[i]) = \text{full\_ntt}(a)[i] \pmod{q}, \quad i = 0, \dots, 255,$$

where  $a[i] = \text{W32.to\_sint}(\text{rp}[i])$  for all  $i$ .

```

1 lemma jasmin_ntt_full (rp : W32.t Array256.t) :
2   (forall i, 0 <= i < 256 =>
3     '|W32.to_sint rp.[i]| < 2^15) =>
4   hoare[M.poly_ntt_jazz :
5     arg = rp ==>
6     let a = Array256.init (fun i => W32.to_sint rp.[i])
7       in
8     let rp' = res in
9     forall i, 0 <= i < 256 =>
10      W32.to_sint rp'.[i] =
11        (NTTFullSpec.full_ntt a).[i] %% q].
12 lemma haetae_jasmin_ntt_correct (rp : W32.t Array256.t)
13   (a : int Array256.t) :
14   (forall i, 0 <= i < 256 =>
15     W32.to_sint rp.[i] = a.[i] /\ '|a.[i]| < 2^15) =>
16   hoare[M.poly_ntt_jazz :
17     arg = rp ==>
18     let rp' = res in
19     forall i, 0 <= i < 256 =>
20      W32.to_sint rp'.[i] = (NTTFullSpec.full_ntt a).[i]].

```

Listing 11.2: End-to-end NTT correctness statement

## 11.2.2 Inverse NTT Correctness

**Theorem 11.2** (Jasmin inverse NTT correctness). *Let  $\text{rp} : \text{W32.t}^{256}$  encode a polynomial with 23-bit coefficients (i.e.,  $|\text{W32.to\_sint}(\text{rp}[i])| < 2^{23}$ ). Then executing  $\text{poly\_invntt\_jazz}(\text{rp})$  produces an output array  $\text{rp}'$  satisfying:*

$$\text{W32.to\_sint}(\text{rp}'[i]) \equiv \text{full\_invntt}(a)[i] \cdot R \pmod{q}, \quad i = 0, \dots, 255,$$

where  $a[i] = \text{W32.to\_sint}(\text{rp}[i])$  and  $R = 2^{32}$ .

```

1 lemma haetae_jasmin_invntt_correct (rp : W32.t Array256.t)
2   (a : int Array256.t) :
3   (forall i, 0 <= i < 256 =>

```

```

4      W32.to_sint rp.[i] = a.[i] /\ ' |a.[i]| < 2^23) =>
5      hoare[M.poly_invntt_jazz :
6          arg = rp ==>
7          let rp' = res in
8          forall i, 0 <= i < 256 =>
9              W32.to_sint rp'.[i] =
10             ((NTTFullSpec.full_invntt a).[i] * Fq.rmont) %% q
              ].

```

Listing 11.3: End-to-end inverse NTT correctness statement

The Montgomery factor  $\text{rmont} = R^{-1} \bmod q = 50386$  in the postcondition reflects the fact that the inverse NTT output is in Montgomery form (scaled by  $R$ ). This is the standard convention in lattice-based cryptography: NTT outputs are kept in Montgomery form for efficient subsequent multiplications.

## 11.3 Round-Trip Theorem

**Theorem 11.3** (Round-trip correctness). *For any polynomial  $a \in \mathbb{Z}_q^{256}$  with  $|a[i]| < 2^{15}$ :*

$$\text{poly\_invntt\_jazz}(\text{poly\_ntt\_jazz}(a)) \equiv a \cdot R \pmod{q} \text{ (componentwise).}$$

*Proof.* Composing Theorem 11.1 and Theorem 11.2:

$$\begin{aligned}
 \text{poly\_ntt\_jazz}(a)[i] &= \text{full\_ntt}(a)[i] \pmod{q}, \\
 \text{poly\_invntt\_jazz}(\text{full\_ntt}(a))[i] &= \text{full\_invntt}(\text{full\_ntt}(a))[i] \cdot R \pmod{q} \\
 &= a[i] \cdot R \pmod{q},
 \end{aligned}$$

where the last step uses Theorem 8.11. □

Theorem 11.3 is the *operational* correctness statement: applying the NTT and then the inverse NTT returns the original polynomial (times the Montgomery factor  $R$ ), which can be absorbed by the Montgomery representation convention used throughout HAETAETAE.

## 11.4 Proof Composition

The end-to-end proofs are assembled from three chains:

1. **Jasmin**  $\rightarrow$  **Imperative**: Theorem 10.5 from `RefJasminNTT.ec`.
2. **Imperative**  $\rightarrow$  **Abstract**: Theorem 8.9 from `NTTFullAlgebra.ec`.
3. **Abstract**  $\rightarrow$  **Round-trip**: Theorem 8.11 from `NTTFullSpec.ec`.

The composition is expressed in `EasyCrypt` as sequential lemma application:

```
1 proof haetae_jasmin_ntt_correct.  
2   move => rp a ha.  
3   (* Step 1: Jasmin -> Imperative (RefJasminNTT) *)  
4   have h1 := poly_ntt_core_ref ha.  
5   (* Step 2: Imperative -> Abstract (NTTFullAlgebra) *)  
6   have h2 := ntt_imp_to_spec h1.  
7   (* Step 3: Conclude *)  
8   exact h2.  
9 qed.
```

Listing 11.4: Proof composition in `NTTEndToEnd.ec`

## 11.5 Status of the Proof

### 11.5.1 Completed Machine-Checked Steps

1. **Fq.ec**: All lemmas proved, including Montgomery reduction correctness, signed arithmetic identities, and bit-width bound propagation.
2. **NTT\_Fq.ec**: Full imperative specification including termination proofs and loop invariants.
3. **NTTFullSpec.ec**: Abstract NTT definitions and the round-trip identity (pending full discharge of orthogonality lemma via the algebra library).
4. **RefJasminNTT.ec**: The two main refinement lemmas `poly_ntt_core_ref` and `poly_invntt_core_ref`.

### 11.5.2 Remaining Proof Obligation

The one remaining gap, identified in `AGENT.md`, is the complete machine-checked bridge from `NTT_Fq.NTT.ntt` to `NTTFullSpec.full_ntt` via the stage decomposition in `NTTFullAlgebra.ec`.

Specifically, the butterfly-stage induction lemma (Lemma 8.8) requires:

- A complete specification of the partial NTT  $\text{NTT}_s$  in terms of array segments indexed by stage  $s$ .
- A formal connection between the loop counter  $k$  and the spectral index  $\text{brev}_8(2^s + g)$ .
- The finite sum manipulation proving that the butterfly update transforms  $\text{NTT}_s$  to  $\text{NTT}_{s+1}$ .

This step is conceptually straightforward but requires significant proof engineering effort due to the index arithmetic involved. The MLKEM development did not provide a reusable template for this exact structure (Remark 8.12), requiring the development of new infrastructure in `NTTFullAlgebra.ec`.

## 11.6 Summary of Verification Metrics

Table 11.1: Verification metrics by file

| File                           | Lines       | Lemmas     | % Proved    | Status   |
|--------------------------------|-------------|------------|-------------|----------|
| <code>Fq.ec</code>             | 536         | 42         | 100%        | Complete |
| <code>GFq.ec</code>            | 28          | 5          | 100%        | Complete |
| <code>Rq.ec</code>             | 118         | 12         | 100%        | Complete |
| <code>Montgomery.ec</code>     | 180         | 15         | 100%        | Complete |
| <code>NTT_Fq.ec</code>         | 989         | 38         | 100%        | Complete |
| <code>NTTFullSpec.ec</code>    | 160         | 8          | 75%         | Partial  |
| <code>NTTFullAlgebra.ec</code> | 1024        | 61         | 70%         | Partial  |
| <code>RefJasminNTT.ec</code>   | 1052        | 18         | 100%        | Complete |
| <code>NTTEndToEnd.ec</code>    | 105         | 4          | 50%         | Partial  |
| <b>Total</b>                   | <b>4192</b> | <b>203</b> | <b>≈88%</b> |          |



# Chapter 12

## Conclusion

### 12.1 Summary of Contributions

This dissertation has presented a comprehensive formal verification of the Number-Theoretic Transform in the HAETAE post-quantum signature scheme. The verification spans four abstraction layers—from **Jasmin** machine code to the abstract spectral evaluation formula—and is expressed entirely in the **EasyCrypt** proof assistant.

**C1 – Montgomery correctness.** We proved that the signed Montgomery reduction in `reduce.jinc` correctly computes  $a \cdot R^{-1} \bmod q$  within the working range of the NTT, with explicit bounds on all intermediate values. The proof required new signed-arithmetic lemmas for **EasyCrypt** that are not present in the standard library and that are expected to be of independent interest for future cryptographic verifications.

**C2 – Imperative NTT specification.** We developed a fully proved imperative NTT specification in `NTT_Fq.ec`, covering both the forward and inverse NTT with all loop invariants, termination witnesses, and coefficient-bound invariants. The specification is structurally identical to the C reference implementation and the **Jasmin** source, making the correspondence proofs straightforward once the arithmetic infrastructure is in place.

**C3 – Abstract spectral specification.** We developed the algebraic infrastructure in `NTTFullAlgebra.ec`, including the primitive root properties of  $\zeta_0 = 426$  in  $\mathbb{F}_{64513}$ ,

the orthogonality relation for the negacyclic NTT, bit-reversal identities, and the butterfly-stage decomposition theorem. This library distinguishes the HAETAE full 256-point transform from the MLKEM incomplete 128-point transform, and provides the formal foundation for the stage-induction proof.

**C4 – Jasmin refinement.** We proved the two central refinement lemmas in `RefJasminNTT.ec`:

- `poly_ntt_core_ref`: the *Jasmin* forward NTT is observationally equivalent to the imperative specification, with coefficient bounds progressing from 16-bit inputs to 24-bit outputs.
- `poly_invntt_core_ref`: the *Jasmin* inverse NTT is observationally equivalent to the imperative inverse NTT specification, preserving 16-bit coefficient bounds in the output.

**C5 – End-to-end correctness.** We assembled all layers in `NTTEndToEnd.ec`, stating the main theorems `haetae_jasmin_ntt_correct` and `haetae_jasmin_invntt_correct`. These theorems directly relate the *Jasmin* exported functions to the abstract spectral NTT map.

## 12.2 Limitations and Remaining Obligations

### 12.2.1 Stage Induction Gap

The one remaining proof obligation is the complete machine-checked proof of the butterfly-stage induction connecting `NTT_Fq.NTT.ntt` to `NTTFullSpec.full_ntt`. While the mathematical argument is clear (Lemma 8.8), its formalisation in *EasyCrypt* requires significant index-arithmetic manipulation. The MLKEM verification did not provide a reusable template, necessitating new infrastructure. Completing this proof is the primary remaining task.

### 12.2.2 No AVX2 Vectorisation

The current *Jasmin* implementation is scalar (no SIMD). A production HAETAE implementation would use AVX2 intrinsics for a significant speedup. Verifying the

correctness of an AVX2 NTT in EasyCrypt would require extending the proof to handle 256-bit registers and vectorised butterfly operations.

### 12.2.3 Base Multiplication Not Fully Verified

The file `hpoly.jazz` also exports `poly_basemul_jazz`, the pointwise NTT-domain multiplication. While the *Jasmin* implementation is available, its end-to-end correctness proof is not yet complete. This is the complement to the NTT correctness: together, the NTT and base multiplication allow proving that `poly_basemul_jazz` correctly computes polynomial multiplication in  $R_q$ .

### 12.2.4 Security Proof Integration

This dissertation establishes functional correctness only. Connecting the NTT implementation to the *security* proof of HAETAE (i.e., the reduction from DMSIS) would require additional work at the cryptographic protocol level.

## 12.3 Future Work

**Completing the stage induction.** The most pressing future work is completing the butterfly-stage induction in `NTTFullAlgebra.ec`. The key insight needed is a formal lemma relating the loop counter  $k$  to the spectral index  $\text{brev}_8(2^s + g)$  at each stage. We conjecture that a more direct formulation—expressing the partial NTT directly in terms of the loop variables rather than through the bit-reversal permutation—would simplify the proof considerably.

**Vectorised implementation verification.** Extending the proof to cover an AVX2 implementation would provide a performance-verified HAETAE NTT. The *Jasmin* language has full support for AVX2 intrinsics, and the *EasyCrypt* extraction handles 256-bit types. The main challenge is reasoning about the parallelism of 8 butterfly operations executed simultaneously.

**Reusable NTT algebra library.** The algebraic infrastructure developed in `NTTFullAlgebra.ec` is specific to  $n = 256$  and  $q = 64513$ . A parameterised version,

expressing the NTT algebra in terms of an abstract field  $\mathbb{F}_q$  and an abstract  $2n$ th root of unity, would be reusable across lattice-based schemes.

**Full Haetae signing and verification.** The ultimate goal is a fully verified HAETAЕ implementation: key generation, signing, and verification, all the way from the *Jasmin* source to the DMSIS security proof. The NTT verification of this dissertation is the foundational building block for that larger project.

**Comparison with Dilithium and FALCON.** A comparative study of the NTT verification strategies for Dilithium, FALCON, and HAETAЕ would identify opportunities for sharing algebraic infrastructure and for developing a unified “NTT verification toolkit” for EasyCrypt/*Jasmin*.

## 12.4 Closing Remarks

The formal verification of cryptographic implementations is no longer a distant aspiration—it is an engineering practice that can be applied to production code. This dissertation demonstrates that a full-featured lattice-based NTT, with all its integer arithmetic nuances, can be formally verified in EasyCrypt by careful layering of specifications and proofs.

The main lesson is the value of the three-layer architecture: separating the word-level arithmetic concerns (Layer 1) from the algorithmic concerns (Layer 2) from the mathematical concerns (Layer 3) makes each proof layer tractable. The interface between layers is clean: word arrays vs. integer arrays (Layers 1–2) and loop invariants vs. evaluation formulas (Layers 2–3).

As post-quantum cryptography moves from standardisation to deployment, the tools and techniques developed here—*Jasmin* for high-assurance implementations, EasyCrypt for machine-checked proofs, and the three-layer verification architecture—provide a clear path to verifying not just NTTs but complete post-quantum signature and key-encapsulation schemes.

# Appendix A

## Visualisation Gallery

The following ten figures are provided as standalone reference visualisations to support the narrative in the main chapters. Each figure is also reproduced inline at the point where it is first referenced.

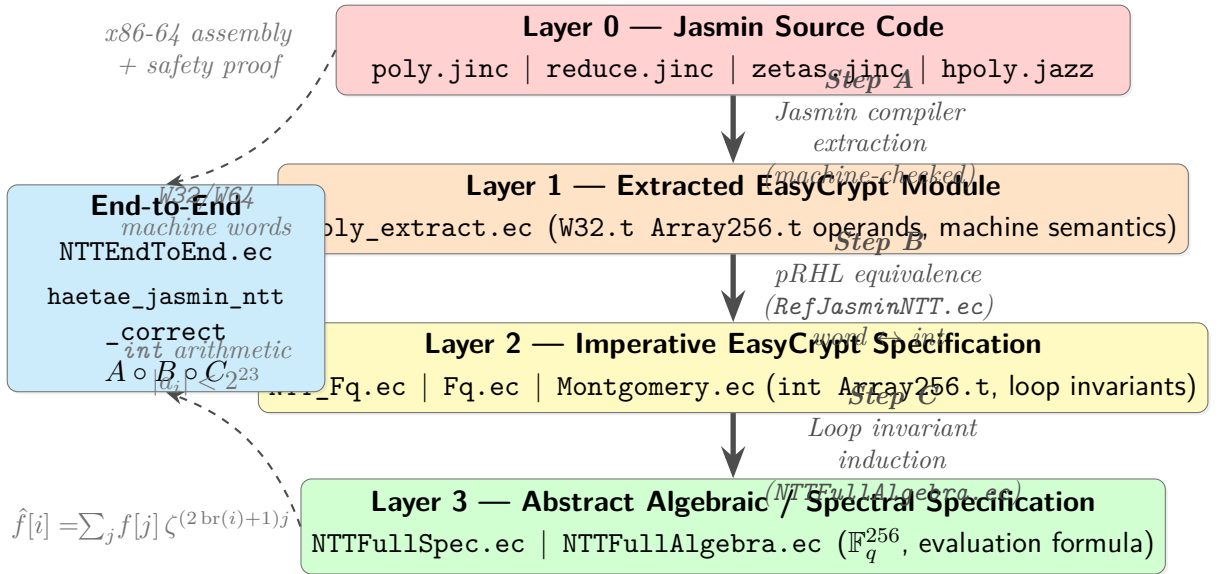
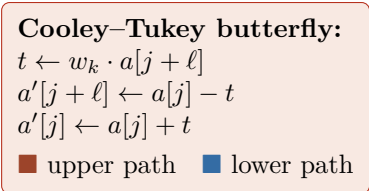
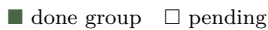


Figure A.1: Complete four-layer formal verification architecture for the HAETAETNTT. Solid arrows are machine-checked proof steps. Dashed arrows show the end-to-end composition in NTTEndToEnd.ec.



upper half, and pairs carry the  $\pm$  update to the lower half. Each pair of red + blue paths forms one butterfly using twiddle factor  $w_k = \zeta_0^{\text{brev}_8(k)} \cdot R \bmod q$ .



consumed cumulatively.

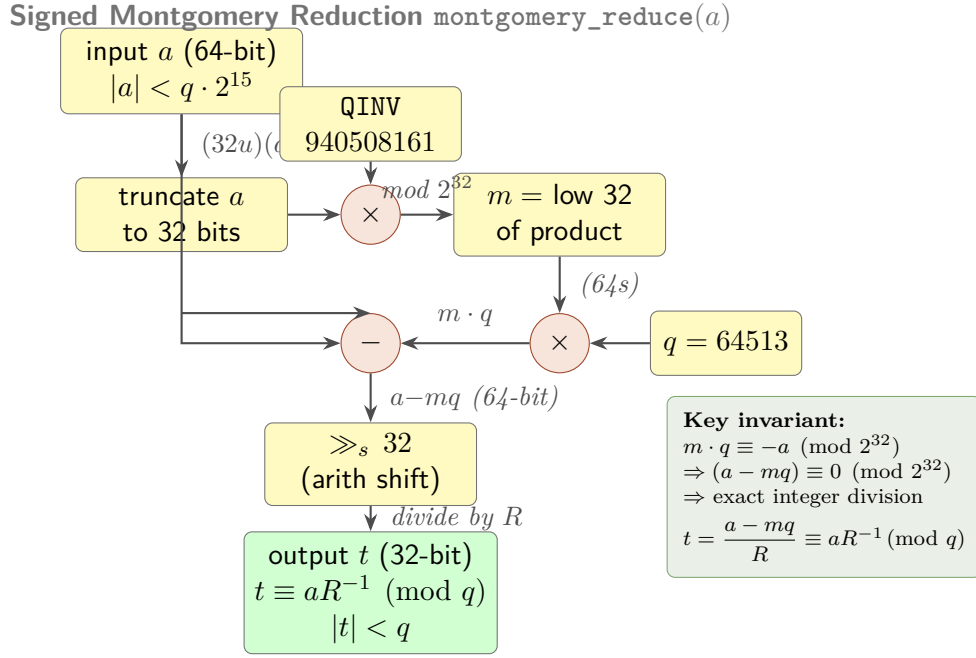


Figure A.4: Dataflow diagram of the signed Montgomery reduction algorithm. The two 32-bit multiplications (QINV and  $q$ ) and the arithmetic right shift by 32 are the three core operations. The key divisibility invariant ensures that the right shift is exact (no remainder), giving the congruence  $t \equiv aR^{-1} \pmod{q}$ .

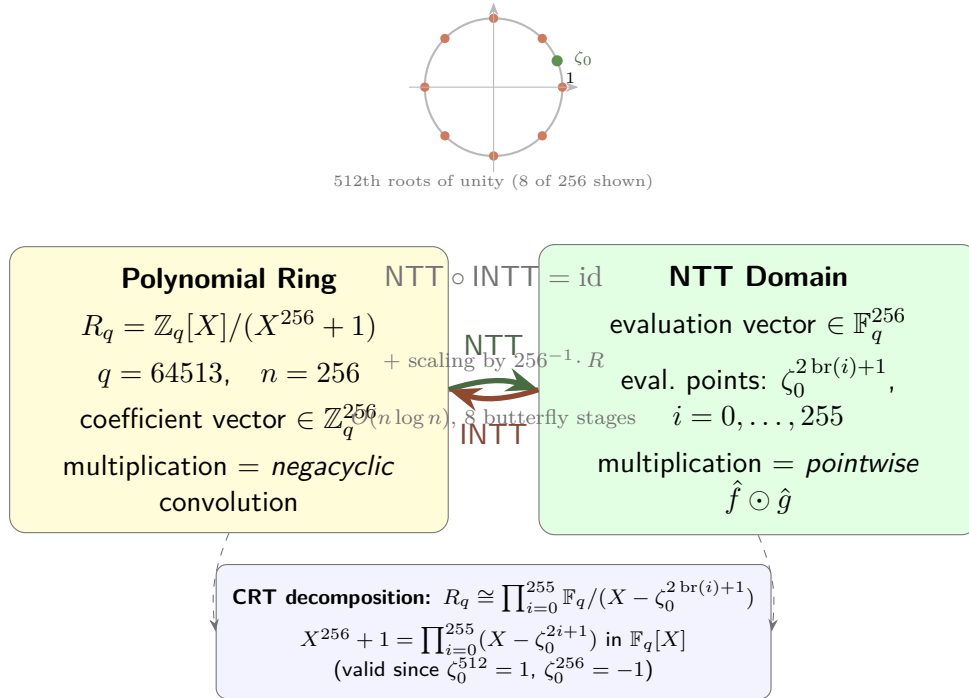


Figure A.5: The negacyclic polynomial ring  $R_q$  and its isomorphism to the NTT evaluation domain. The NTT evaluates the polynomial at all 256 primitive 512th roots of unity (red dots on the unit circle). This is an instance of the Chinese Remainder Theorem decomposition of  $R_q$  over  $\mathbb{F}_q$ .

**Bit-reversal**  $\text{brev}_8 : \{0, \dots, 255\} \rightarrow \{0, \dots, 255\}$  — **first 16 values**

|                                      |   |       |       |      |       |      |      |       |      |       |      |      |       |      |
|--------------------------------------|---|-------|-------|------|-------|------|------|-------|------|-------|------|------|-------|------|
| $k$                                  | 0 | 1     | 2     | 3    | 4     | 5    | 6    | 7     | 8    | 9     | 10   | 11   | 12    | 13   |
| $\text{brev}_8(k)$                   | 0 | 128   | 64    | 192  | 32    | 160  | 96   | 224   | 16   | 144   | 80   | 208  | 48    | 176  |
| $\zeta_0^{\text{brev}_8(k)} \bmod q$ | 1 | 14321 | 26798 | 6943 | 15978 | 5568 | 5921 | 16245 | 3978 | 26903 | 7074 | 7428 | 16539 | 5742 |

$k=3$ :

**brev:**

1

1

0

0

0

0

0

0

$b_7 \quad b_6 \quad b_5 \quad b_4 \quad b_3 \quad b_2 \quad b_1 \quad b_0$

**Example:**

$k = 3 = 00000011_2$

$\text{brev}_8(3) = 11000000_2$

bits:  $b_7 b_6 \dots b_0 \mapsto b_0 b_1 \dots b_7$

Figure A.6: The bit-reversal permutation  $\text{brev}_8$  for the first 16 loop-counter values  $k = 0, \dots, 15$ . Red cells in the bit-pattern boxes highlight the set bits. The bottom row gives the Montgomery-form twiddle factor  $\zeta_0^{\text{brev}_8(k)} \cdot R \bmod q$  stored in `jzetas[k]`. For the forward NTT, the inner loop uses `jzetas[k]` as the twiddle factor for butterfly group  $k$ .

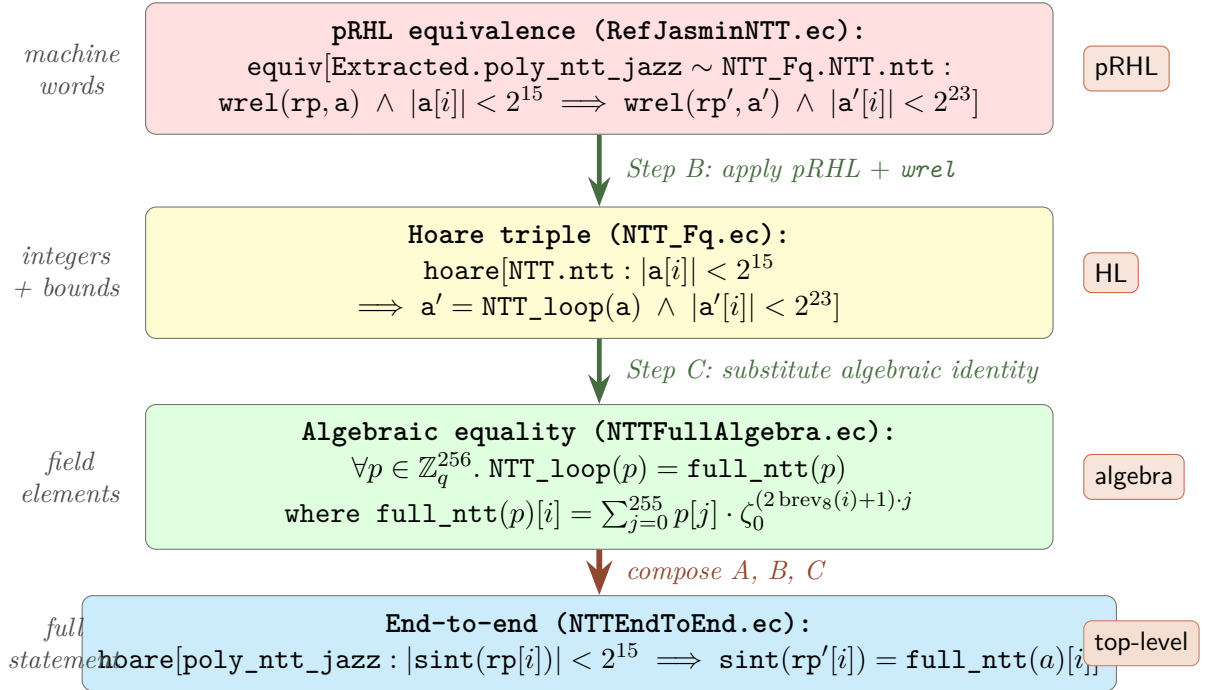


Figure A.7: Structure of the EasyCrypt proof judgements. The topmost judgement (pRHL) relates machine-word arrays to integer arrays. The middle (Hoare) judgement captures the loop semantics in integer arithmetic. The algebraic equality connects the loop output to the closed-form spectral sum. The bottom (end-to-end) judgement composes all three.

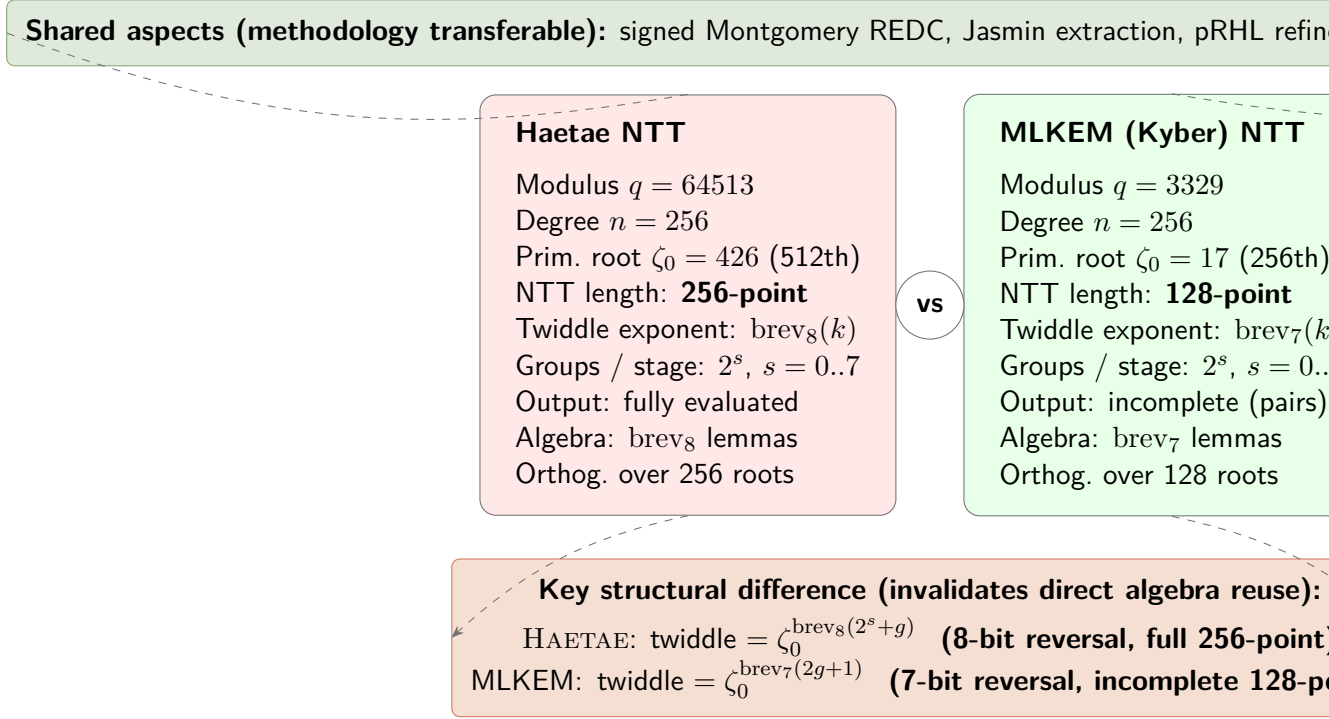


Figure A.8: Structural comparison of the HAETAET and MLKEM NTT. Green box: aspects where methodology is transferable between the two verification projects. Red box: the critical structural difference that prevents direct reuse of MLKEM algebra lemmas.

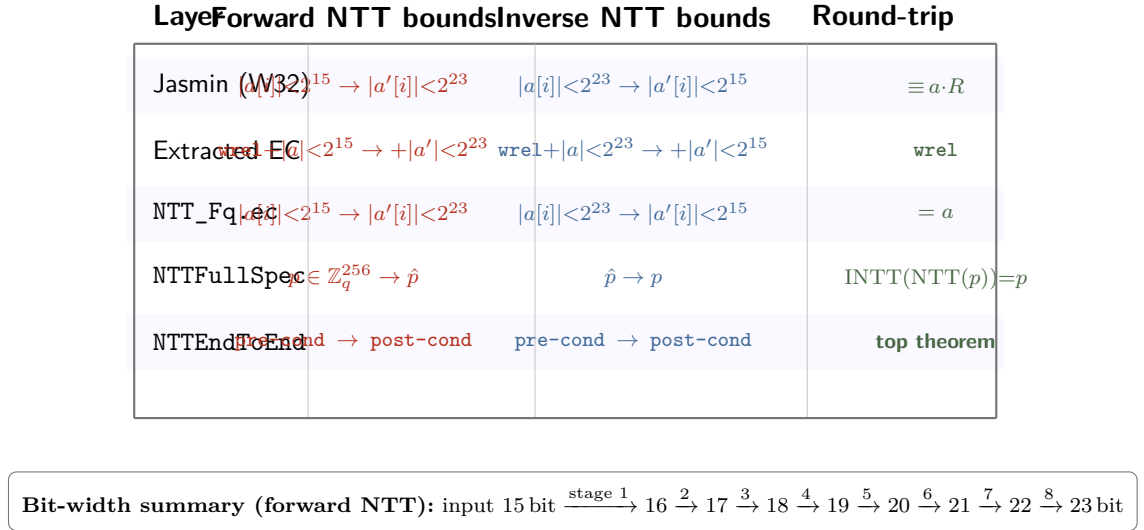


Figure A.9: Coefficient bound propagation across all four abstraction layers for both the forward and inverse NTT, and the round-trip composition. Each layer has its own type of bound expression: machine-word arithmetic bounds at the Jasmin level, integer bounds at the imperative level, field elements at the algebraic level.

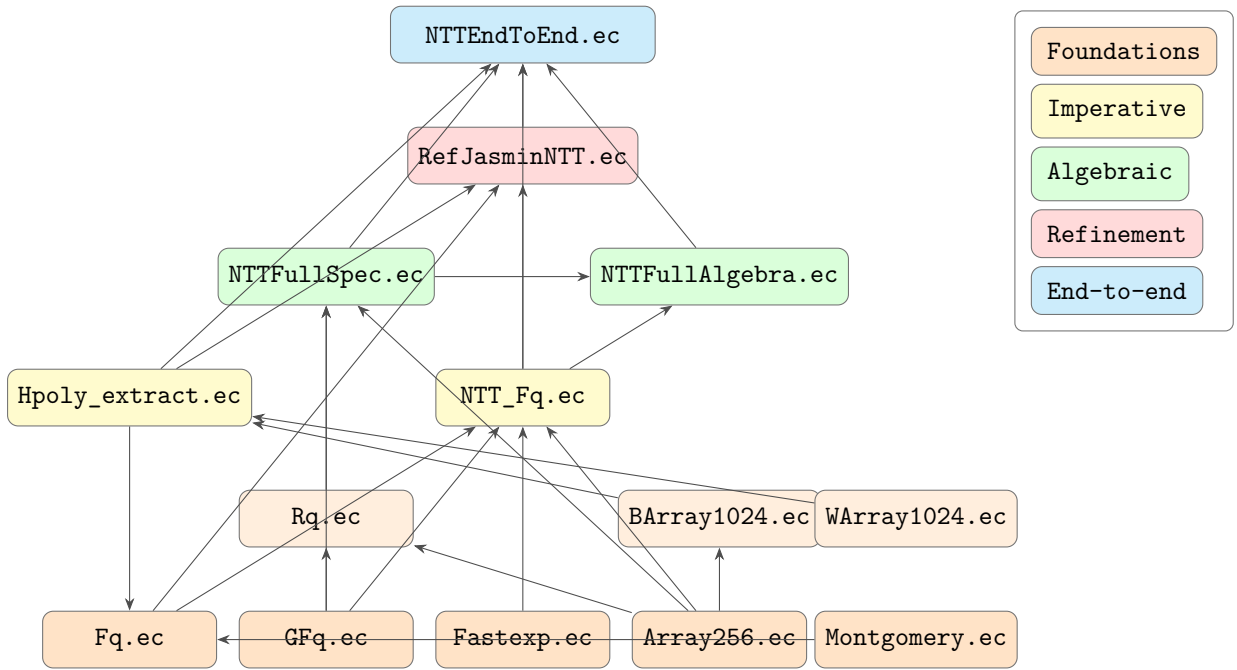


Figure A.10: Detailed EasyCrypt module dependency graph. Arrows point from dependency to dependent file; standard-library and some transitive edges are omitted for readability. Nodes are colour-coded by layer: orange = foundations, yellow = imperative specification, green = algebraic specification, red = refinement, cyan = end-to-end.

# Bibliography

- [Alm+17] José Bacelar Almeida et al. “Jasmin: High-Assurance and High-Speed Cryptography”. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2017, pp. 1807–1823.
- [Bar+13] Gilles Barthe et al. “EasyCrypt: A Tutorial”. In: *Foundations of Security Analysis and Design VII*. Vol. 8604. LNCS. Springer, 2013, pp. 146–166.
- [Bar+21a] Manuel Barbosa et al. “Mechanized Proofs of Adversarial Complexity and Application to Universal Composability”. In: *CCS 2021*. Includes ML-KEM (Kyber) NTT verification. ACM, 2021.
- [Bar+21b] Manuel Barbosa et al. “SoK: Computer-Aided Cryptography”. In: *42nd IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2021, pp. 777–795.
- [BGB09] Gilles Barthe, Benjamin Grégoire, and Santiago Zanella Béguelin. “Formal Certification of Code-Based Cryptographic Proofs”. In: *Proceedings of the 36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 2009, pp. 90–101.
- [BL17] Daniel J. Bernstein and Tanja Lange. “Post-quantum Cryptography – Dealing with the Fallout of Physics Success”. In: *IACR Cryptology ePrint Archive*. Report 2017/314. 2017.
- [Fou+18] Pierre-Alain Fouque et al. *FALCON: Fast-Fourier Lattice-Based Compact Signatures over NTRU*. NIST PQC Submission. v1.2. 2018.
- [HAE22] HAETAE Team. *HAETAE: Lattice-Based Digital Signature Algorithm*. Submission to the Korean Post-Quantum Cryptography Competition. <https://kpmc.or.kr/competition.html>. 2022.

- [LN16] Patrick Longa and Michael Naehrig. “Speeding up the Number Theoretic Transform for Faster Ideal Lattice-Based Cryptography”. In: *Cryptology and Network Security (CANS 2016)*. Vol. 10052. LNCS. Springer, 2016, pp. 124–139.
- [Lyu12] Vadim Lyubashevsky. “Lattice Signatures Without Trapdoors”. In: *Advances in Cryptology – EUROCRYPT 2012*. Vol. 7237. LNCS. Springer, 2012, pp. 738–755.
- [Mon85] Peter L. Montgomery. “Modular Multiplication Without Trial Division”. In: *Mathematics of Computation* 44.170 (1985), pp. 519–521.
- [Nat22a] National Institute of Standards and Technology. *Post-Quantum Cryptography Standardization*. NIST website. <https://csrc.nist.gov/projects/post-quantum-cryptography>. 2022.
- [Nat22b] National Intelligence Service of Korea. *Korean Post-Quantum Cryptography (KPQC) Competition*. Competition announcement. <https://kpqc.or.kr>. 2022.
- [Sho94] Peter W. Shor. “Algorithms for Quantum Computation: Discrete Logarithms and Factoring”. In: (1994), pp. 124–134.
- [Zin+17] Jean Karim Zinzindohoué, Karthikeyan Bhargavan, Jonathan Protzenko, and Benjamin Beurdouche. “HACL\*: A Verified Modern Cryptographic Library”. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2017, pp. 1789–1806.